

DESIGN AND EVALUATION OF HIGH SPEED ADDERS

A Main Project Report

Submitted to the APJ Abdul Kalam Technological University

in partial fulfillment of requirements for the award of degree

Bachelor of Technology

in

Electronics and Communication Engineering

by

ANJALI PRAKASH P. (PKD22EC14)

ANJANA C.(PKD22EC015)

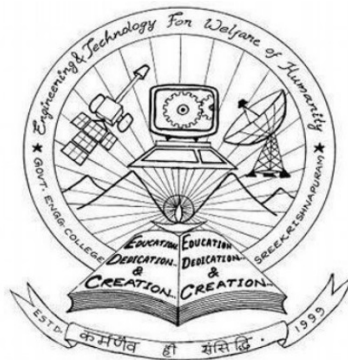
GOWRI MOHAN S.(PKD22EC028)

SHEETAL A. R.(PKD22EC053)

under the Guidance of

Dr. CHITHIRA P. R.

Assistant Professor, Department of ECE



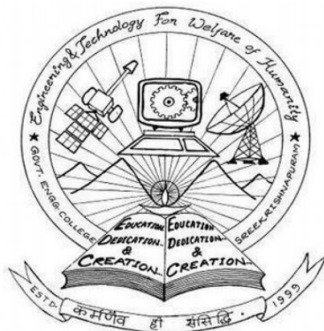
DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

GOVERNMENT ENGINEERING COLLEGE PALAKKAD

SREEKRISHNAPURAM

APRIL 2026

CERTIFICATE



This is to certify that the report entitled **DESIGN AND EVALUATION OF HIGH SPEED ADDERS** submitted by **ANJALI PRAKASH P.(PKD22EC014)**, **ANJANA C.(PKD22EC015)**, **GOWRI MOHAN S.(PKD22EC028)**, **SHEETAL A. R.(PKD22EC053)** to the APJ Abdul Kalam Technological University in partial fulfillment of the B.Tech. degree in Electronics and Communication Engineering is a bonafide record of the Project Phase I carried out by him/her under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Dr. Chithira P. R.
(Project Guide)
Assistant Professor
Dept. of ECE
Govt. Engineering College
Palakkad

Prof. Jithesh C. P.
(Project Coordinator)
Assistant Professor
Dept. of ECE
Govt. Engineering College
Palakkad

Dr. Chithira P. R.
(Project Coordinator)
Assistant Professor
Dept. of ECE
Govt. Engineering College
Palakkad

Dr. Mohanan K. P.
(Head of the Department)
Associate Professor
Dept. of ECE
Govt. Engineering College
Palakkad

ABSTRACT

High-speed adders play a crucial role in modern digital systems where arithmetic operations must be executed efficiently. Conventional adders such as the Ripple Carry Adder (RCA) suffer from large propagation delays, while faster architectures like Kogge–Stone and Han–Carlson offer improved performance at the cost of increased wiring complexity and area. This project focuses on the design and evaluation of a Hybrid Carry Tree Adder (HCA–Brent Kung) to achieve a balanced trade-off between speed, area, and wiring efficiency. The proposed design combines the compact structure of the Brent–Kung Adder for local carry generation and the Han–Carlson prefix logic for global carry computation, thereby optimizing both delay and layout complexity. The work involves the simulation and implementation of RCA, Brent–Kung, Han–Carlson, and the proposed Hybrid Carry Tree Adder using the Vivado Design Suite 2024.1, followed by a detailed performance analysis based on area utilization, power consumption, delay, and routing complexity.

ACKNOWLEDGEMENT

First of all I thank the Almighty, for giving me the strength ,courage and knowledge to do this project successfully.

I express my heartfelt gratitude to the Principal **Dr. K. R. Ramesh Babu**, Govt Engineering College, Sreekrishnapuram, for his inspiration throughout the project.

I would also like to thank **Dr. MOHANAN K.P.**, Head Of the Department, Electronics and Communication Engineering, Govt. Engineering College, Palakkad for providing permission and availing all required facilities for undertaking the project in a systematic way.

I am extremely grateful to **Dr. CHITHIRA P. R.**, Assistant Professor, Govt. Engineering College, Palakkad, who guided me with kind, cordial, valuable suggestions.I would like to appreciate the guidance given by **Prof. JITHESH C. P.** Assistant Professor and **Dr. CHITHIRA P. R.** Assistant Professor my project coordinators,Department of Electronics and Communication Engineering, Govt. Engineering College, Palakkad, for providing sincere guidance.

I would also like to express my sincere gratitude to all teaching and non-teaching staff of Department of Electronics and Communication Engineering, Govt. Engineering College, Palakkad, for the sincere directions imparted and the cooperation in connection with the project.

I am also thankful to my parents for the support given in connection with the project. Gratitude may be extended to all well-wishers and my friends who supported me for the project.

Contents

Abstract	i
Acknowledgement	ii
List of Figures	iv
List of Tables	v
List of Symbols	vi
1 INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Aim	2
1.4 Objective	2
1.5 SCOPE	3
1.6 Overview of the Project	3
2 LITERATURE REVIEW	5
3 PROBLEM DEFINITION	8
4 DESIGN METHODOLOGY	10
4.1 Overview	10
4.2 Step 1: Review of Existing Adder Architectures	11
4.3 Step 2: Identification of Design Limitations	11
4.4 Step 3: Implementation of Existing Adders	11
4.5 Step 4: Synthesis and Performance Analysis	11
4.6 Step 5: Comparative Evaluation of Existing Architectures	12

4.7	Step 6: Design of Proposed Hybrid Carry Tree Adder	12
4.8	Step 7: Performance Comparison and Verification	14
4.9	Step 8: Performance Analysis of Proposed Design	15
5	ARCHITECTURE USED	16
5.1	OVERVIEW	16
5.2	Prefix Cells: Black Cell and Grey Cell	16
5.3	BRENT-KUNG ADDER (BKA)	18
5.4	HAN-CARLSON ADDER (HCA)	20
5.5	PROPOSED HYBRID CARRY TREE ARCHITECTURE	22
5.5.1	Pre-processing Stage	22
5.5.2	Bit Partitioning Strategy	23
5.5.3	Prefix Computation Stage	23
5.5.4	Carry Generation Stage	23
5.5.5	Post-processing Stage	24
5.5.6	Overall Working of Hybrid Architecture	24
6	RESULTS	27
6.1	SIMULATION ENVIRONMENT	27
6.2	SIMULATION RESULTS	27
6.3	PERFORMANCE ANALYSIS	32
7	CONCLUSION	34
7.1	Limitations	35
7.2	Future Work	35
	References	36
	Appendix	37
.0.1	Black Cell Code	37
.0.2	Grey Cell code	37
.0.3	Source code for 8 bit bka adder	37
.0.4	Test bench code for 8 bit bka adder	39
.0.5	Source code for 8 bit hca adder	41
.0.6	Test bench code for 8 bit hca adder	43

.0.7	Source code for 8 bit hybrid adder	44
.0.8	Test bench code: 8 bit hybrid adder	46
.0.9	Source code for 16 bit hybrid adder	48
.0.10	Test bench code: 16 bit hybrid adder	51
.0.11	Source Code: 32 bit hybrid adder	52
.0.12	Test bench code: 32 bit hybrid adder	59

List of Figures

4.1	Flowchart of Design Methodology	10
4.2	Design Flow of Proposed Hybrid Carry Tree Adder	14
5.1	8 bit Brent kung adder structure	18
5.2	16 Brent kung adder structure	19
5.3	8 bit Han-carlson structure.	20
5.4	16 bit Han-carlson structure.	21
5.5	8 bit Hybrid structure.	24
5.6	16-bit Hybrid structure	25
5.7	32-bit Hybrid structure	26
6.1	Timing diagram of 8 bit BKA.	28
6.2	Timing diagram of 16 bit BKA.	29
6.3	Timing diagram of 8 bit HCA.	29
6.4	Timing diagram of 16 bit HCA.	30
6.5	Timing diagram of 8 bit HYBRID.	30
6.6	Timing diagram of 16 bit HYBRID.	31
6.7	Timing diagram of 32 bit HYBRID.	31

List of Tables

2.1	Comparison of Different Adder Architectures	6
6.1	Simulation Environment Details	27
6.2	Performance Comparison of BKA, HCA and Hybrid Adders	32
6.3	Theoretical Analysis of Delay	33

List of Symbols

- A, B Input operands
 S Sum output
 C Carry output
 P Propagate signal ($P = A \oplus B$)
 G Generate signal ($G = A \cdot B$)
 C_{in} Input carry
 C_{out} Output carry
 n Number of bits
 G_k Generate signal at stage k
 P_k Propagate signal at stage k
 G_j Generate signal at stage j
 P_j Propagate signal at stage j
 $G_{k:j}$ Group generate signal from bit j to k
 $P_{k:j}$ Group propagate signal from bit j to k

Chapter 1

INTRODUCTION

1.1 Background

The rapid advancement of digital systems, including microprocessors, digital signal processors (DSPs), and VLSI circuits, has significantly increased the demand for high-speed arithmetic operations. Among all arithmetic units, adders play a fundamental role, as they are extensively used in operations such as addition, multiplication, and address computation.

The performance of an entire digital system is highly dependent on the efficiency of its adder circuits. Key performance parameters such as propagation delay, area utilization, power consumption, and routing complexity are directly influenced by the design of the adder. As modern applications require faster processing speeds and compact hardware, optimizing adder architectures has become a critical area of research.

Traditional adders like the Ripple Carry Adder (RCA) are simple and area-efficient but suffer from large propagation delays. To overcome this limitation, advanced parallel prefix adders such as Brent–Kung (BKA), Kogge–Stone, and Han–Carlson (HCA) have been developed. These architectures improve speed by computing carry signals in parallel, but they introduce trade-offs in terms of area and wiring complexity.

Therefore, designing an optimized adder that balances speed, area, and complexity remains a key challenge in modern digital design.

1.2 Problem Statement

Despite the availability of various adder architectures, achieving an optimal balance between speed, area, and routing complexity remains a major challenge. The Ripple Carry Adder (RCA), although simple and area-efficient, suffers from high propagation delay due to

sequential carry generation.

On the other hand, parallel prefix adders such as Han–Carlson and Kogge–Stone significantly reduce delay by enabling parallel carry computation. However, these architectures require a large number of interconnections and logic elements, leading to increased area utilization and routing complexity. Brent–Kung adder offers a compromise with reduced wiring and area, but it is comparatively slower than other parallel prefix adders.

Thus, there exists a need for an efficient adder architecture that combines the advantages of high-speed operation and reduced hardware complexity. The challenge lies in designing an adder that minimizes delay while also optimizing area and interconnect requirements for FPGA-based implementations.

1.3 Aim

The aim of this project is to design and evaluate a high-speed hybrid carry tree adder by combining the advantages of Brent–Kung and Han–Carlson architectures to achieve an optimal trade-off between delay, area, and routing complexity.

1.4 Objective

The objectives of this project are:

- To study and analyze different adder architectures such as Ripple Carry Adder (RCA), Brent–Kung Adder (BKA), and Han–Carlson Adder (HCA) in Vivado Design Suite software.
- To design and implement 8-bit, 16-bit, and 32-bit versions of HCA , BKA adders using Verilog HDL.
- To evaluate the performance of each adder in terms of propagation delay, area utilization, and wiring complexity.
- To develop a hybrid carry tree adder that combines the advantages of Brent–Kung and Han–Carlson architectures.
- Achieve lower area and wiring complexity with a balanced trade-off between speed and area
- To compare the performance of the proposed hybrid adder with existing architectures and identify improvements.

1.5 SCOPE

This project focuses on the design and evaluation of high-speed adder architectures for FPGA-based systems. The implementation is carried out using Verilog HDL and simulated using Xilinx Vivado Design Suite 2024.1, targeting the Artix-7 FPGA platform.

The study includes the analysis of Ripple Carry, Brent–Kung, and Han–Carlson adders, followed by the development of a hybrid carry tree adder. The evaluation is based on key performance parameters such as delay, area utilization, and routing complexity.

However, the scope of this project is limited to simulation and FPGA-based implementation. Power analysis and ASIC-level optimization are not extensively covered. The proposed design can be further extended for low-power applications and large-scale VLSI systems in future work.

1.6 Overview of the Project

This project focuses on the design and evaluation of high-speed adder architectures for modern digital systems. Adders form the core of arithmetic and logic units in microprocessors, digital signal processors (DSPs), and VLSI circuits, where high-speed arithmetic operations are essential.

Conventional adders such as the Ripple Carry Adder (RCA) are simple and area-efficient but suffer from large propagation delays due to sequential carry generation. To address this limitation, parallel-prefix adders like Brent–Kung (BKA) and Han–Carlson (HCA) have been developed, which improve speed by computing carry signals in parallel. However, these architectures introduce trade-offs in terms of increased wiring complexity and area utilization.

In this project, a Hybrid Carry Tree Adder combining Brent–Kung and Han–Carlson architectures is proposed to achieve a balanced trade-off between speed, area, and routing complexity. The design is implemented using Verilog HDL and simulated in Xilinx Vivado Design Suite 2024.1, targeting FPGA-based systems.

Various adder architectures, including RCA, BKA, and HCA, are analyzed and compared based on parameters such as delay, area utilization, power consumption, and routing efficiency. The proposed hybrid design demonstrates improved performance by reducing delay while maintaining efficient resource utilization, making it suitable for high-speed and resource-

constrained digital applications.

Chapter 2

LITERATURE REVIEW

The design of high-speed adders has been a significant area of research in digital electronics due to the increasing demand for faster and more efficient arithmetic operations in modern computing systems. Numerous studies have been conducted to optimize adder performance in terms of propagation delay, area utilization, and power consumption. However, it is widely recognized that no single adder architecture can simultaneously achieve optimal performance across all these parameters. As a result, various hybrid and modified adder structures have been proposed to balance these conflicting design requirements.

Kousalya et al. reported that the Han–Carlson adder provides an effective balance between speed and area due to its hybrid prefix structure, which reduces the number of logic levels without significantly increasing hardware complexity. Their study also highlighted that the Kogge–Stone adder, although achieving very low delay, requires a large number of interconnections, leading to higher routing complexity and power consumption. In contrast, the Brent–Kung adder was found to be the most area-efficient architecture, making it well suited for FPGA-based implementations where resource utilization is a critical factor [1].

S. K. Yezerla et al. demonstrated that the Brent–Kung adder achieves a favorable delay–area trade-off among various parallel-prefix adders. While Han–Carlson and Kogge–Stone adders offer slightly better delay performance, they incur significantly higher area and routing overhead. The study also observed that power consumption remains relatively similar across these architectures, suggesting that optimization efforts should primarily focus on reducing delay and area. These findings support the development of hybrid architectures that combine the speed advantages of Han–Carlson with the area efficiency of Brent–Kung [2].

Anjana D et al. analyzed the performance of the Kogge–Stone adder and concluded that it achieves the minimum propagation delay due to its fully parallel prefix structure. The authors emphasized the importance of developing optimized or hybrid prefix structures that can retain

high-speed performance while reducing hardware requirements. This insight directly aligns with the objective of the present work, which aims to reduce redundant prefix operations while maintaining fast carry computation [3].

Senthil Ganesh R et al. evaluated various adders in terms of delay, area, and power consumption using CMOS technology. Their results indicated that the Han–Carlson adder offers better overall performance and power efficiency compared to the Kogge–Stone adder, primarily due to reduced interconnect complexity. Although the Kogge–Stone adder achieves the lowest delay, it suffers from increased power dissipation due to its extensive wiring and high fan-out. The Han–Carlson adder, therefore, provides a practical trade-off for high-speed applications. Furthermore, hybrid structures based on Han–Carlson architecture have been proposed to further optimize delay and area. The hybrid Han–Carlson adder presented in [6] demonstrates improved performance, reinforcing its suitability for hybrid adder design [4].

G. Chandrasekhar et al. reported that the Brent–Kung adder, owing to its logarithmic carry tree structure and reduced fan-out, achieves excellent area and power efficiency while maintaining competitive delay performance. The study further emphasized that its lower interconnect complexity results in reduced routing delay, making it highly suitable for FPGA-based systems. Similar observations regarding high-speed parallel prefix adder design are also reported in [8]. These characteristics make Brent–Kung an ideal candidate [5].

Table 2.1: Comparison of Different Adder Architectures

Adder	Speed	Area	Complexity	Limitation
Kogge-Stone	Very High	High	Very High	High wiring complexity
Han-Carlson	High	Medium	Medium	Moderate routing complexity
Brent-Kung	Medium	Low	Low	Slightly higher delay
Ripple Carry	Low	Very Low	Very Low	Very high delay

From Table 2.1, it is evident that each adder exhibits specific trade-offs between speed, area, and complexity. This further justifies the need for a hybrid architecture that balances these parameters effectively.

From the above studies, it can be concluded that Han–Carlson adders provide superior speed but at the cost of increased hardware complexity and routing overhead, whereas Brent–Kung adders offer better area efficiency with slightly higher delay. Kogge–Stone adders, although achieving the best delay performance, are often impractical for FPGA implementations due to excessive wiring and power requirements. Therefore, combining the advantages of Han–Carlson and Brent–Kung architectures presents a promising approach for designing a

balanced and efficient adder. This hybrid strategy aims to minimize delay while controlling area and routing complexity, forming the basis of the proposed work.

Chapter 3

PROBLEM DEFINITION

Adders are one of the most fundamental components in digital systems, particularly in arithmetic and logic units (ALUs), digital signal processors (DSPs), and other high-performance computing circuits. The overall speed, area, and power consumption of a system are highly influenced by the performance of its adder circuit. Therefore, designing an adder that offers high speed, low area utilization, and reduced routing complexity is a critical design challenge. Several adder architectures have been developed to balance these factors, but each has specific limitations.

- Ripple Carry Adder (RCA) is the simplest adder design, where the carry output of each stage propagates sequentially to the next stage. Although easy to implement and area-efficient, its major drawback is the long carry propagation delay, which increases linearly with bit size. Hence, RCA becomes inefficient for high-bit operations.
- Brent–Kung Adder (BKA) improves upon RCA by using a parallel prefix structure that computes carries in a tree-like manner. It offers a good balance between speed and area, as it reduces carry delay logarithmically. However, while Brent–Kung is area-efficient, it is not the fastest among parallel prefix adders due to its longer prefix depth compared to advanced architectures like Kogge–Stone or Han–Carlson.
- Han–Carlson Adder (HCA), on the other hand, combines the features of Kogge–Stone and Brent–Kung adders to achieve higher speed and lower logic levels. The HCA significantly reduces propagation delay by utilizing more parallel computation paths. However, this improvement comes at the cost of increased wiring complexity and area utilization, as it requires more interconnects and logic resources.

It is evident that RCA suffers from poor speed, while Han–Carlson, though faster, consumes

more area and routing resources. Brent–Kung provides an intermediate solution with better area efficiency but moderate delay.

This hybrid design aims to achieve a trade-off between speed and area, maintaining the fast computation capability of the Han–Carlson adder while minimizing the wiring complexity and resource usage by integrating Brent–Kung’s optimized structure. In essence, the proposed Hybrid Brent Kung–Han Carlson Adder adopts the low-fanout and balanced tree structure of the Brent–Kung architecture in the initial stages, where local generate and propagate signals are computed efficiently using fewer interconnects. This significantly reduces the number of logic resources and routing paths required. In the subsequent stages, the design leverages the high-speed carry generation of the Han–Carlson structure, which employs additional parallel prefix nodes to accelerate global carry computation. By strategically combining these two design philosophies, the hybrid adder achieves reduced prefix levels, fewer interconnects and wiring segments compared to Han–Carlson and optimized resource utilization.

Chapter 4

DESIGN METHODOLOGY

The design methodology adopted in this project follows a structured and systematic approach for analyzing, implementing, and optimizing various adder architectures on an FPGA platform. The primary objective is to develop a hybrid carry tree adder that combines the area efficiency of the Brent–Kung adder (BKA) and the high-speed performance of the Han–Carlson adder (HCA). The overall methodology is divided into the following stages.

4.1 Overview

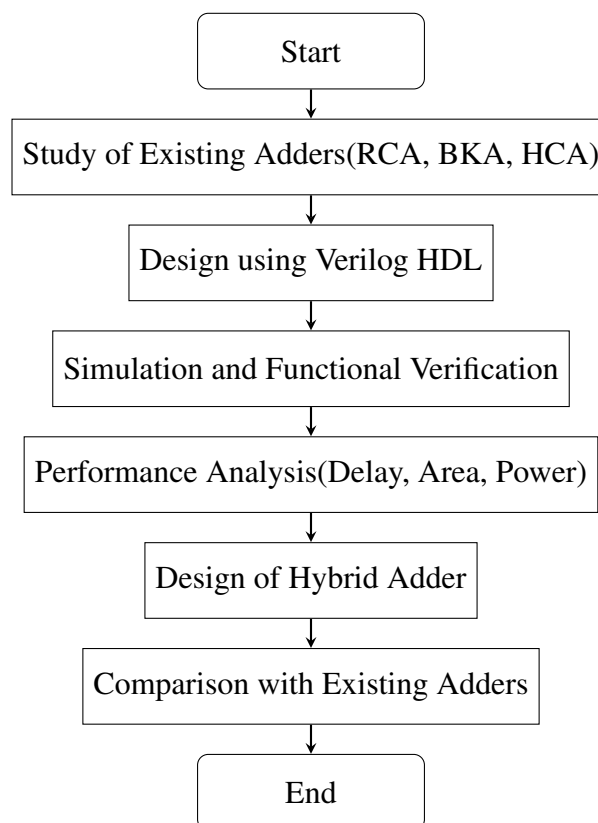


Figure 4.1: Flowchart of Design Methodology

4.2 Step 1: Review of Existing Adder Architectures

The first step involves an extensive study of conventional and parallel-prefix adders, including Ripple Carry Adder (RCA), Brent–Kung Adder (BKA), and Han–Carlson Adder (HCA). Each architecture is analyzed in terms of propagation delay, hardware complexity, and area utilization.

From this analysis, it is observed that RCA is simple but suffers from large delay, BKA provides good area efficiency with moderate speed, and HCA offers high-speed performance at the cost of increased routing complexity. These observations highlight the need for a hybrid approach.

4.3 Step 2: Identification of Design Limitations

Based on the literature review and initial analysis, the limitations of each adder architecture are identified. RCA suffers from sequential carry propagation, BKA has increased logic depth compared to other prefix adders, and HCA requires more hardware resources and interconnections.

This step establishes the design requirement for an optimized architecture that can reduce delay while maintaining low area and routing complexity.

4.4 Step 3: Implementation of Existing Adders

To establish a baseline for comparison, 8-bit, 16-bit, and 32-bit versions of Brent–Kung and Han–Carlson adders are implemented using Verilog HDL in Xilinx Vivado Design Suite 2024.1. The target device selected is Xilinx Artix-7 (xc7a35tcp236-1).

Each design is synthesized and simulated to verify functional correctness. This ensures that all architectures operate accurately before proceeding to performance evaluation.

4.5 Step 4: Synthesis and Performance Analysis

After implementation, synthesis and implementation reports are generated and analyzed. The following parameters are evaluated:

- Hardware utilization (LUTs, slice registers)
- Timing performance (maximum propagation delay)
- Routing characteristics (number of nets and interconnections)

The analysis reveals that Brent–Kung uses fewer resources with moderate delay, whereas Han–Carlson achieves lower delay at the expense of higher routing complexity and resource usage.

4.6 Step 5: Comparative Evaluation of Existing Architectures

The performance of BKA and HCA is compared for different bit-widths. It is observed that BKA provides better area efficiency, while HCA offers superior speed. Additionally, scalability analysis shows that resource utilization and delay increase with bit-width, particularly for HCA.

This comparison highlights the inherent trade-off between speed and area, forming the basis for hybrid design.

4.7 Step 6: Design of Proposed Hybrid Carry Tree Adder

Based on the insights obtained from the analysis of existing architectures, a hybrid carry tree adder is designed by combining Brent–Kung and Han–Carlson prefix structures to achieve an optimal balance between delay, area, and routing complexity.

The proposed design utilizes:

- Brent–Kung structure in the initial stages to reduce area and interconnect complexity
- Han–Carlson structure in the later stages to improve carry propagation speed

Bit Partitioning Strategy: The N-bit adder (8-bit, 16-bit, or 32-bit) is partitioned into smaller groups, typically 4-bit blocks. The lower-order blocks are implemented using Brent–Kung logic, while the higher-order blocks use Han–Carlson logic.

This partitioning is based on the observation that lower-order bits lie in non-critical paths and can be optimized for area and wiring, whereas higher-order bits lie in critical paths and require faster carry propagation.

Hybrid Prefix Structure: The hybrid prefix tree is constructed by combining sparse and dense prefix computation. Brent–Kung logic is used in early stages to minimize the number of prefix nodes and interconnections, while Han–Carlson logic is applied in later stages to reduce logic depth and improve speed.

Use of Black and Grey Cells: The design of the hybrid prefix tree involves careful placement of black and grey cells to optimize both delay and hardware utilization. Black cells are used in intermediate stages where both generate and propagate signals are required, while grey cells are used in final stages to compute carry outputs efficiently by eliminating unnecessary propagate calculations.

Design Rationale: The hybrid approach is based on the trade-off between speed and hardware complexity. Brent–Kung reduces fan-out and wiring complexity, while Han–Carlson reduces propagation delay through parallel computation. By combining these two architectures, the hybrid adder achieves a balanced carry computation path.

This approach ensures reduced delay in critical paths, efficient hardware utilization, and improved scalability for higher bit-width designs.

The hybrid structure is implemented for 8-bit, 16-bit, and 32-bit configurations to evaluate scalability and performance across different bit-widths. This approach ensures a balanced carry computation path, optimizing both delay and hardware utilization.

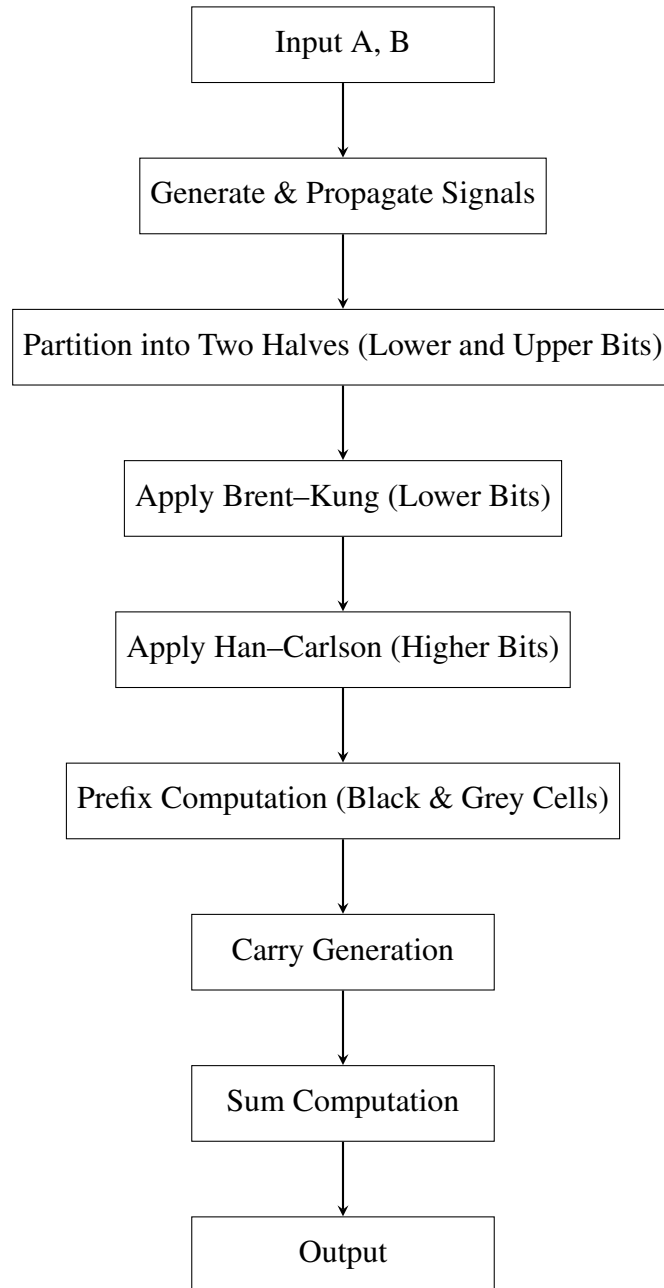


Figure 4.2: Design Flow of Proposed Hybrid Carry Tree Adder

4.8 Step 7: Performance Comparison and Verification

The hybrid adder is implemented and evaluated alongside BKA and HCA using the same simulation environment. The comparison is carried out based on:

- Area (slice utilization)
- Routing complexity (number of nets)
- Propagation delay

4.9 Step 8: Performance Analysis of Proposed Design

The hybrid adder is analyzed for different bit-widths, and the results indicate that:

- Hybrid-8 achieves comparable delay with reduced area
- Hybrid-16 achieves the lowest delay with balanced resource usage
- Hybrid-32 shows reduced routing complexity with improved efficiency

Overall, the hybrid adder demonstrates reduced hardware utilization, balanced delay performance, and improved efficiency compared to conventional architectures.

The observed variations in delay are influenced by synthesis and routing optimizations performed by the Vivado tool. However, the results closely align with theoretical expectations.

Chapter 5

ARCHITECTURE USED

5.1 OVERVIEW

The project aims to design and evaluate high-speed adder architectures using Verilog HDL in Xilinx Vivado 2024.1. The implemented adders include:

- Brent–Kung Adder (BKA)
- Han–Carlson Adder (HCA)
- Hybrid Adder (HCA -BKA)

A comparative analysis of these architectures was conducted to determine an optimized structure that minimizes area, wiring complexity and delay. All designs were simulated using the Vivado simulator, targeting the Artix-7 FPGA (xc7a35tcp236-1, Speed Grade – 1) [7].

5.2 Prefix Cells: Black Cell and Grey Cell

Parallel prefix adders utilize fundamental building blocks known as black cells and grey cells to efficiently compute carry signals [7].

Generate and Propagate Signals

For each bit position i , the generate and propagate signals are defined as:

$$G_i = A_i \cdot B_i$$

$$P_i = A_i \oplus B_i$$

Black Cell

A black cell computes both the group generate and group propagate signals. It combines two pairs of generate and propagate inputs.

Inputs: (G_k, P_k) and (G_j, P_j)

$$G_{k:j} = G_k + (P_k \cdot G_j)$$

$$P_{k:j} = P_k \cdot P_j$$

Grey Cell

A grey cell computes only the group generate signal.

Inputs: (G_k, P_k) and G_j

$$G_{k:j} = G_k + (P_k \cdot G_j)$$

Since propagate signals are not required in the final stages of carry computation, grey cells help reduce hardware complexity.

Role in Prefix Adders

Black and grey cells are arranged in a tree-like structure to compute carries in parallel.

Advantages

- Enables parallel carry computation, reducing delay
- Minimizes logic depth to $\log_2 N$
- Reduces wiring complexity using structured prefix trees
- Grey cells reduce area by eliminating unnecessary propagate computations
- Improves overall speed compared to ripple carry adders

5.3 BRENT-KUNG ADDER (BKA)

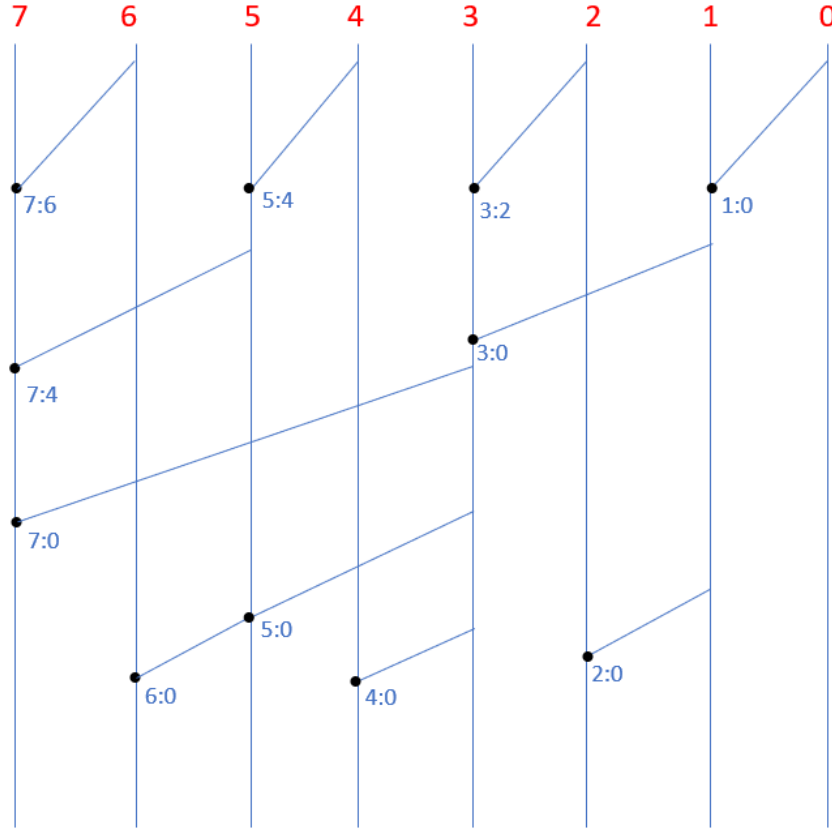


Figure 5.1: 8 bit Brent kung adder structure

Figure 5.1 shows the 8-bit Brent–Kung Adder structure. The prefix network is organized into a logarithmic tree with a limited number of stages, specifically following a depth of approximately $2 \log_2(n) - 1$. For an 8-bit implementation, this results in fewer prefix levels, making the structure relatively compact.

The carry computation is divided into two phases: prefix computation and carry distribution. In the prefix stage, black cells are used to combine generate and propagate signals hierarchically, forming intermediate group generate signals. In the second stage, grey cells are employed to distribute the final carry outputs to each bit position efficiently.

Due to the reduced number of nodes and restricted fan-out, the design achieves low wiring complexity and area efficiency. However, the sequential dependency between prefix stages introduces a slightly longer critical path compared to more parallel architectures. Overall, the 8-bit BKA demonstrates an efficient balance between hardware utilization and delay, making it suitable for area-constrained designs.

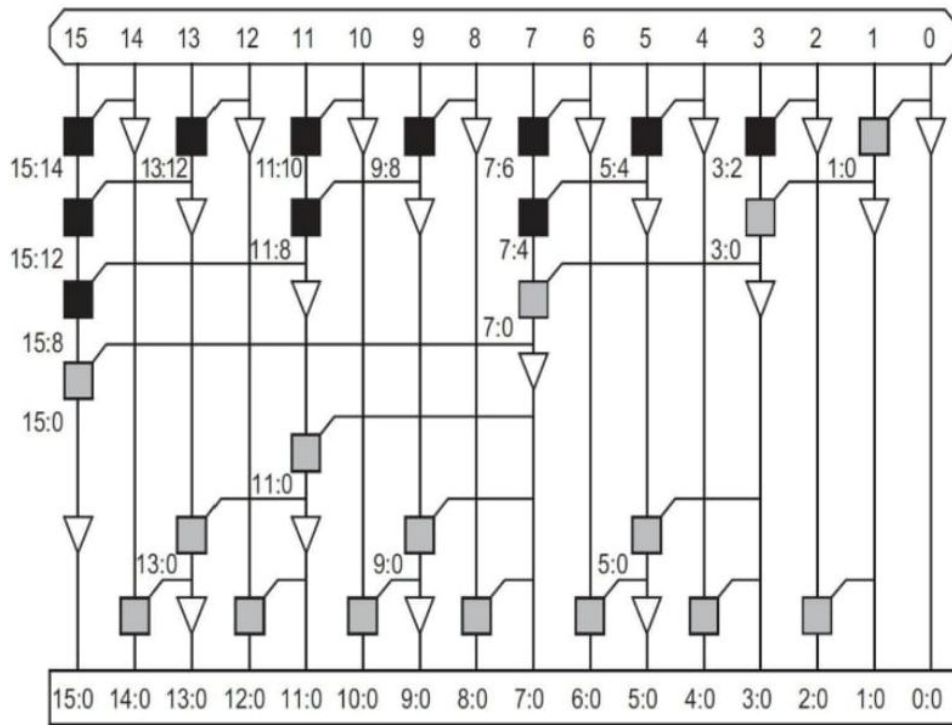


Figure 5.2: 16 Brent kung adder structure

The figure 5.2 shows the Brent–Kung Adder (BKA) structure, adapted from [1], which is a parallel-prefix adder that provides an efficient trade-off between propagation delay and hardware complexity. It follows a tree-based prefix structure designed to minimize fan-out and interconnect wiring, making it highly suitable for FPGA implementations.

Structure: The Brent–Kung adder consists of two main phases: the prefix computation stage and the carry distribution stage. Several modified versions of the Brent–Kung adder have been proposed to further improve speed while maintaining low area. The work in [8] also demonstrates a high-speed parallel prefix adder design with improved delay performance, reinforcing its suitability for efficient arithmetic design. In the prefix stage, generate and propagate signals are combined hierarchically using black cells to compute intermediate carry signals. In the distribution stage, grey cells are used to propagate the final carry values to each bit position. The structure is symmetric and balanced, reducing wiring complexity.

Working: Initially, generate (G_i) and propagate (P_i) signals are computed for each bit. These signals are then combined in a tree structure to compute group generate signals. The computation proceeds in logarithmic stages, where each stage combines results from previous stages. Finally, the carry signals are distributed back to compute the sum outputs.

Advantages:

- Low area utilization due to fewer prefix nodes
- Reduced wiring complexity and fan-out
- Suitable for FPGA implementations

Disadvantages:

- Slightly higher delay compared to faster adders like Kogge–Stone
- Increased logic depth compared to Han–Carlson

Complexity: The logic depth of the Brent–Kung adder is approximately $2 \log_2(n) - 1$, where n is the number of bits. This results in moderate delay while maintaining efficient hardware utilization.

5.4 HAN–CARLSON ADDER (HCA)

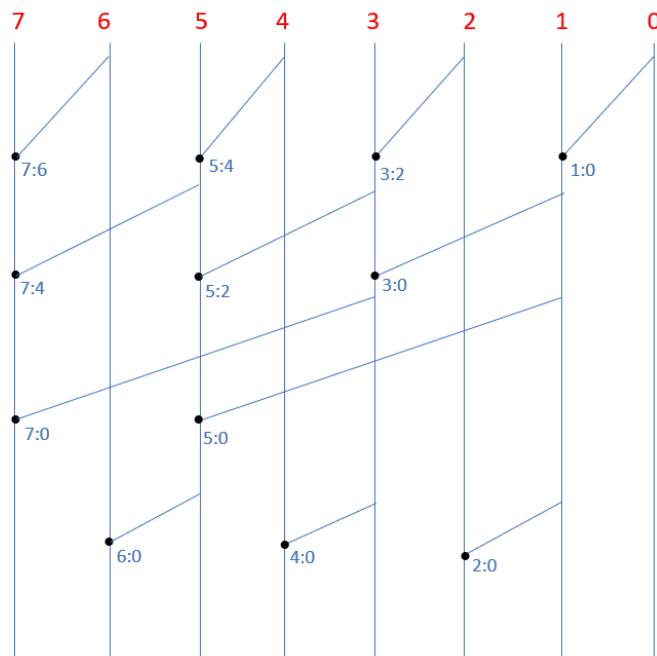


Figure 5.3: 8 bit Han-carlson structure.

Figure 5.3 shows the 8-bit Han–Carlson Adder structure. The prefix network follows a hybrid configuration that combines the fast carry generation of Kogge–Stone in the initial stages with the reduced complexity of Brent–Kung in the later stages.

For the 8-bit case, the number of prefix levels is small, but the architecture still benefits from early parallel carry computation. The first stage rapidly generates partial carry signals using black cells, while subsequent stages selectively reduce the number of nodes using a Brent–Kung style structure. Grey cells are used in the final stage to compute the required carry outputs efficiently.

This hybrid arrangement reduces the overall logic depth to approximately $\log_2(n) + 1$, resulting in faster carry propagation compared to Brent–Kung. Although the improvement is modest at 8 bits due to the already small depth, the architecture still demonstrates better critical path performance while maintaining moderate wiring complexity. This makes the 8-bit HCA a good compromise between speed and hardware overhead.

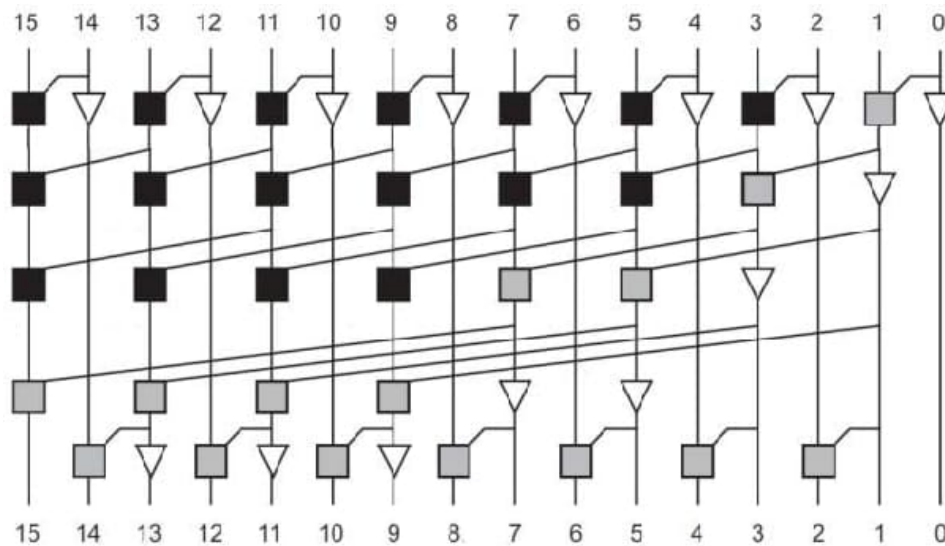


Figure 5.4: 16 bit Han-carlson structure.

Figure 5.4 represents the 16-bit Han–Carlson Adder (HCA) prefix structure, adapted from [1], which combines the advantages of both Kogge–Stone and Brent–Kung adders. The hybrid Han–Carlson structure presented in [6] demonstrates improved speed with reduced logic depth, while similar observations are also reported in [4]. It achieves high speed while maintaining moderate hardware complexity.

Structure: The Han–Carlson adder uses a mixed prefix structure where the initial stages resemble a Kogge–Stone configuration for fast carry generation, while later stages adopt a Brent–Kung style to reduce wiring complexity. Black cells are used for full prefix operations, and grey cells are used to compute final carry outputs efficiently.

Working: The generate and propagate signals are first computed for each bit. In the early stages, carry signals are generated rapidly using parallel computation paths. In later stages, the

structure reduces the number of nodes by selectively combining signals. This results in faster carry propagation with fewer interconnections compared to Kogge–Stone.

Advantages:

- High-speed performance due to reduced logic depth
- Balanced trade-off between speed and area
- Lower wiring complexity compared to Kogge–Stone

Disadvantages:

- Higher area compared to Brent–Kung
- Increased routing complexity

Complexity: The logic depth of the Han–Carlson adder is approximately $\log_2(n) + 1$, making it faster than Brent–Kung while maintaining moderate hardware usage.

5.5 PROPOSED HYBRID CARRY TREE ARCHITECTURE

The proposed Hybrid Carry Tree Adder combines the advantages of Brent–Kung and Han–Carlson parallel prefix architectures. A similar hybrid Han–Carlson approach has been presented in [6], which demonstrates improved delay performance by combining different prefix structures. The design leverages the area efficiency of Brent–Kung in the initial stages and the high-speed carry computation capability of Han–Carlson in the later stages.

The working of the hybrid adder follows the fundamental stages of a parallel prefix adder, namely pre-processing, prefix computation, and post-processing.

5.5.1 Pre-processing Stage

In this stage, the generate and propagate signals are computed for each bit position. These signals form the basis for carry computation.

$$[G_i = A_i \cdot B_i][P_i = A_i \oplus B_i]$$

The generate signal indicates whether a carry is produced at that bit position, while the propagate signal determines whether an incoming carry will be forwarded to the next stage.

5.5.2 Bit Partitioning Strategy

To optimize performance, the N-bit adder (8-bit, 16-bit, or 32-bit) is partitioned into two equal blocks. The lower block handles the lower-order bits using the Brent–Kung structure to minimize hardware complexity and interconnects, while the higher block processes the higher-order bits using the Han–Carlson structure to improve carry propagation speed.

This selective partitioning ensures that critical paths are optimized for speed while non-critical paths are optimized for area efficiency.

5.5.3 Prefix Computation Stage

The prefix computation stage is responsible for calculating group generate and propagate signals using a hierarchical tree structure.

The recursive prefix equations are given by:

$$[G_{i:j} = G_i + (P_i \cdot G_{i-1:j})][P_{i:j} = P_i \cdot P_{i-1:j}]$$

These equations are implemented using black and grey cells arranged in a structured prefix tree.

Black Cell Operation: $[G_{out} = G_k + (P_k \cdot G_j)][P_{out} = P_k \cdot P_j]$

Grey Cell Operation: $[G_{out} = G_k + (P_k \cdot G_j)]$

Black cells compute both generate and propagate signals and are used in intermediate stages, whereas grey cells compute only generate signals and are used in the final stages to reduce hardware complexity.

The hybrid prefix tree applies dense computation (Han–Carlson style) in critical paths to minimize delay, while sparse computation (Brent–Kung style) is used in other regions to reduce area and wiring complexity.

5.5.4 Carry Generation Stage

Using the computed group generate and propagate signals, the carry signals are generated as:

$$[C_{i+1} = G_i + (P_i \cdot C_i)]$$

For group operations:

$$[C_i = G_{i-1:0} + (P_{i-1:0} \cdot C_0)]$$

This stage ensures fast and parallel computation of carry signals across all bit positions.

5.5.5 Post-processing Stage

In the final stage, the sum bits are computed using the propagate signals and the generated carry signals:

$$[S_i = P_i \oplus C_i]$$

This stage completes the addition process by producing the final sum outputs.

5.5.6 Overall Working of Hybrid Architecture

Initially, generate and propagate signals are computed for all input bits. These signals are then processed through a hybrid prefix tree structure, where Brent–Kung logic is used in early stages to minimize interconnections, and Han–Carlson logic is used in later stages to accelerate carry propagation.

By combining dense and sparse prefix computations strategically, the hybrid adder achieves reduced propagation delay, lower hardware utilization, and minimized routing complexity. This makes the proposed architecture highly efficient for FPGA-based high-speed arithmetic applications.

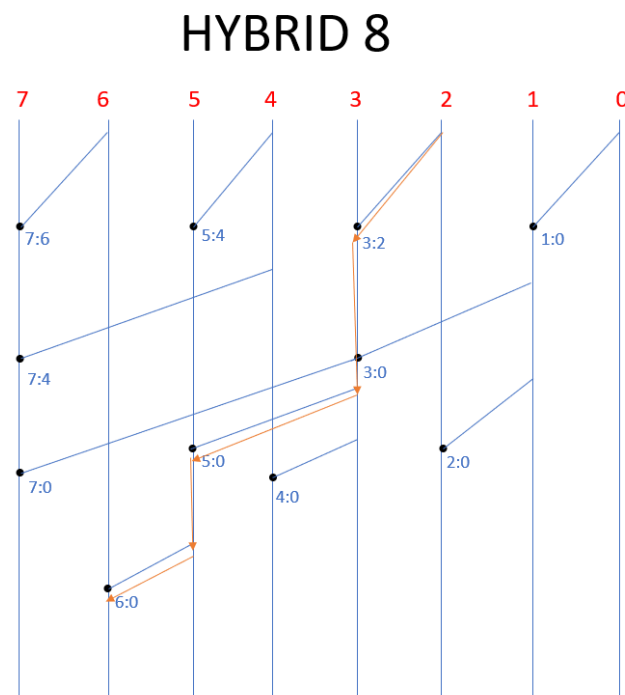


Figure 5.5: 8 bit Hybrid structure.

architecture is designed by partitioning the 8-bit adder into smaller groups, where the lower-order bits are processed using the Brent–Kung prefix structure and the higher-order bits are handled using the Han–Carlson structure.

The Brent–Kung portion reduces wiring complexity and limits fan-out in the initial stages, while the Han–Carlson portion accelerates carry propagation in the later stages. The prefix tree shows a combination of sparse and dense connections, where critical paths utilize more parallel computation to reduce delay.

This hybrid arrangement results in efficient carry computation with reduced hardware overhead, making the 8-bit design suitable for small-scale high-speed arithmetic operations.

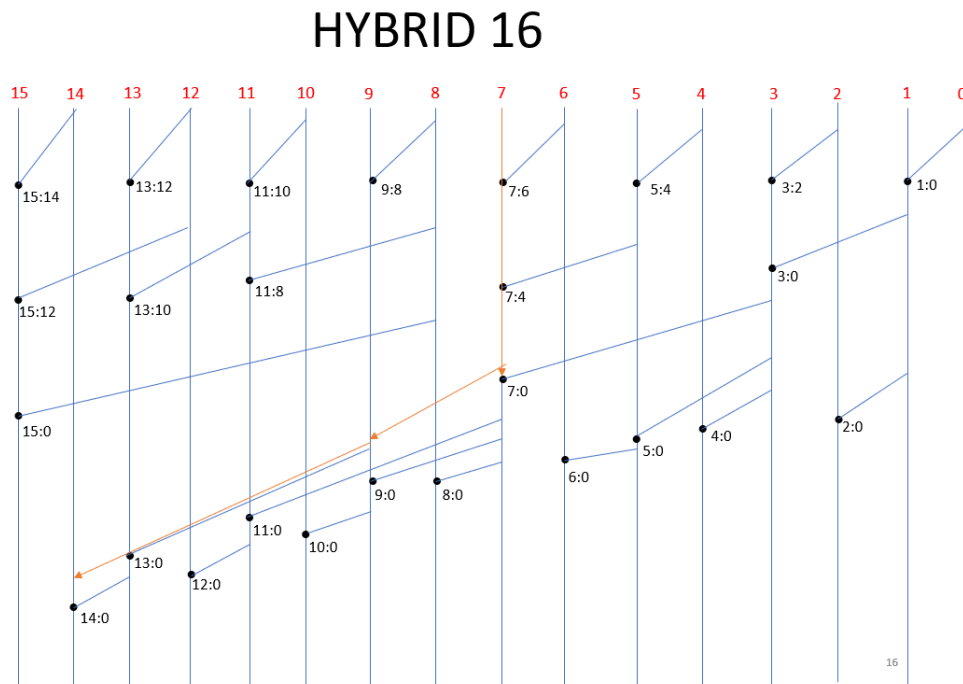


Figure 5.6: 16-bit Hybrid structure

From Figure 5.4, the 16-bit Hybrid Carry Tree Adder demonstrates a more complex prefix structure compared to the 8-bit design. The adder is partitioned into multiple groups, typically 4-bit blocks, where the lower blocks follow Brent–Kung logic and the higher blocks employ Han–Carlson logic.

As the bit-width increases, the number of prefix levels also increases. However, the hybrid structure ensures that delay does not increase significantly by applying faster prefix computation in critical paths. The intermediate nodes illustrate the hierarchical combination of generate and propagate signals using black and grey cells.

This structure achieves a balance between delay and area by reducing unnecessary interconnections in non-critical regions while maintaining fast carry propagation across the entire adder.

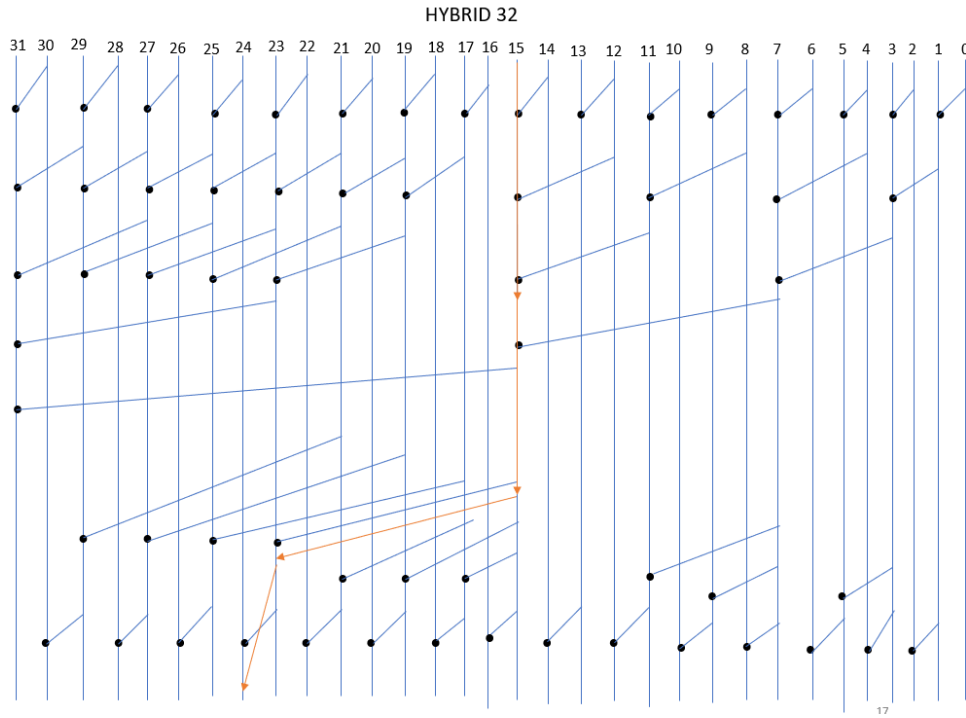


Figure 5.7: 32-bit Hybrid structure

From Figure 5.5, the 32-bit Hybrid Carry Tree Adder exhibits a deep and wide prefix structure due to increased bit-width. The architecture is divided into multiple groups, where Brent–Kung logic is used extensively in the initial stages to minimize wiring complexity, and Han–Carlson logic is applied in higher stages to improve speed.

The prefix tree clearly shows hierarchical carry computation, where group generate signals are computed progressively across stages. The use of a hybrid approach significantly reduces the number of long interconnections compared to a fully parallel prefix adder like Kogge–Stone.

Despite the increase in bit-width, the hybrid design maintains controlled logic depth and efficient resource utilization, demonstrating scalability and suitability for high-performance FPGA-based systems.

The above figures demonstrate that as the bit-width increases, the complexity of the prefix structure also increases. However, the proposed hybrid architecture effectively manages this complexity by combining Brent–Kung and Han–Carlson structures, resulting in improved scalability, reduced delay, and optimized hardware utilization.

Chapter 6

RESULTS

6.1 SIMULATION ENVIRONMENT

Table 6.1: Simulation Environment Details

Parameter	Description
HDL Language	Verilog
Simulation Tool	Xilinx Vivado 2024.1
Target Device	Artix-7 (xc7a35tcpg236-1, Speed Grade – 1, Package – cpga236)
Simulation Type	Behavioral and Post-Synthesis
Operating System	Windows 10
System Configuration	Intel Core i5 CPU, 8 GB RAM

Table 6.1 provides the configuration details of the simulation environment used for designing and verifying the adders. All designs were implemented and tested using Verilog HDL in Xilinx Vivado 2024.1 targeting the Artix-7 FPGA device. The specifications listed ensure a consistent comparison of performance parameters such as area, delay, and power across all architectures.

6.2 SIMULATION RESULTS

The simulation results of Brent–Kung Adder (BKA), Han–Carlson Adder (HCA), and the proposed Hybrid Carry Tree Adder were obtained using Verilog HDL in Xilinx Vivado 2024.1. Timing waveforms were analyzed for 8-bit, 16-bit, and 32-bit implementations to evaluate carry propagation behavior and overall delay performance.

The timing diagrams clearly illustrate the differences in carry computation strategies among the three architectures. In the Brent–Kung adder, carry signals propagate through a structured

but relatively deeper prefix tree, resulting in moderate delay. The Han–Carlson adder improves performance by introducing additional parallel prefix levels, reducing the critical path delay.

The proposed Hybrid Adder combines these characteristics by utilizing Brent–Kung logic in the lower stages and Han–Carlson logic in the higher stages. This results in faster carry propagation in critical paths while maintaining reduced wiring complexity.

From the timing waveforms, it is observed that the Hybrid Adder achieves faster stabilization of sum and carry outputs compared to BKA, and exhibits performance close to or better than HCA, especially for higher bit-widths. This confirms the effectiveness of the hybrid prefix structure in balancing speed and hardware efficiency.

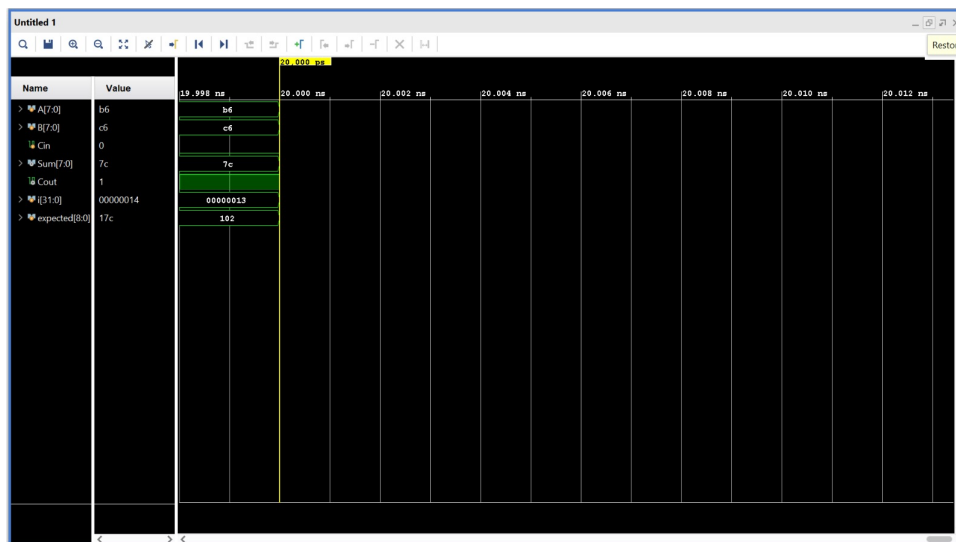


Figure 6.1: Timing diagram of 8 bit BKA.

Figure 6.1 shows the timing waveform of the 8-bit Brent–Kung Adder. The carry signals propagate through a structured prefix tree with fewer levels due to the smaller bit-width. As a result, the delay is relatively low compared to higher bit implementations. However, the sequential nature of carry computation across prefix stages still introduces a moderate propagation delay. The outputs stabilize correctly, verifying the functional accuracy of the design.

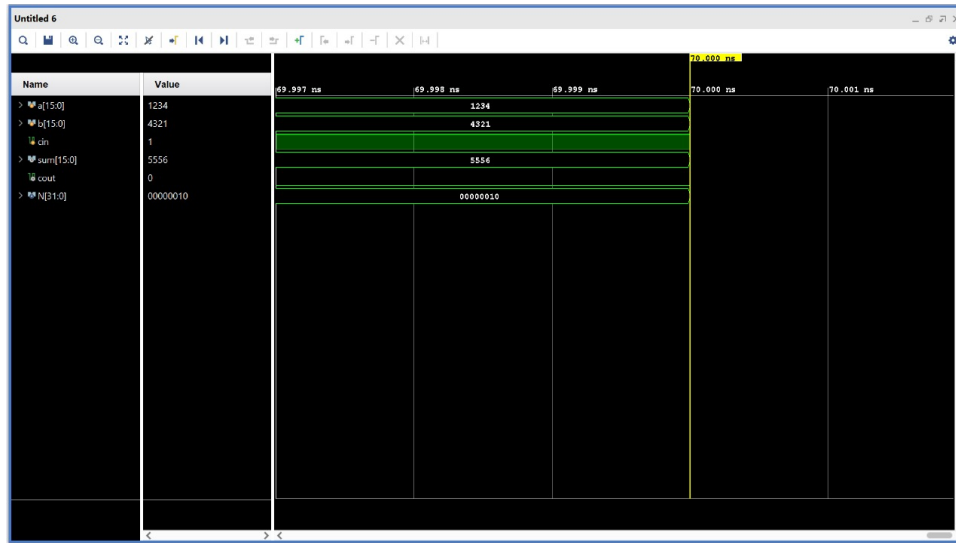


Figure 6.2: Timing diagram of 16 bit BKA.

Figure 6.2 shows the timing waveform of the 16 bit Brent–Kung Adder. The carry signals propagate through multiple prefix levels, resulting in a structured but relatively longer critical path. Although the architecture reduces wiring complexity, the increased logic depth contributes to higher propagation delay.

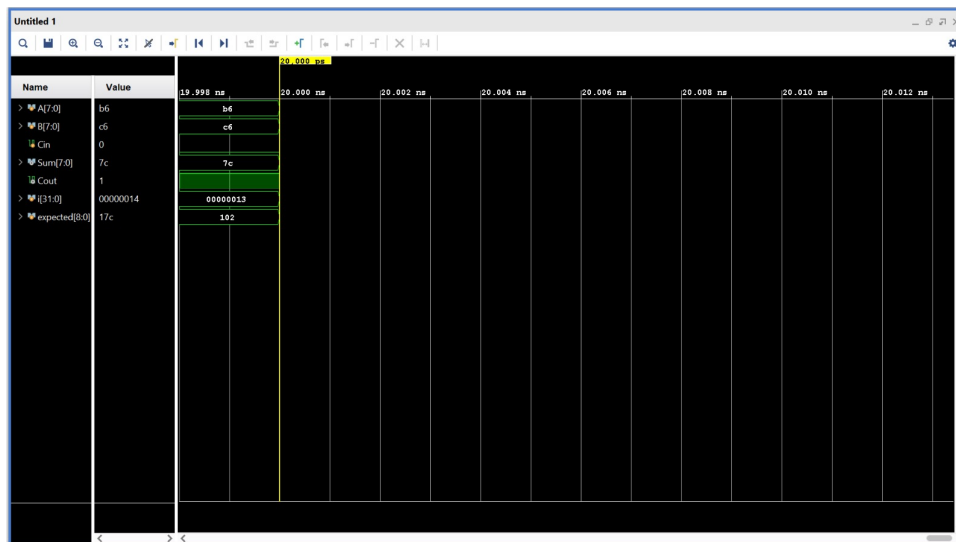


Figure 6.3: Timing diagram of 8 bit HCA.

Figure 6.3 shows the timing waveform of the 8-bit Han–Carlson Adder. The use of parallel prefix computation reduces the number of carry propagation stages, resulting in faster output stabilization compared to the Brent–Kung Adder. Due to the smaller bit-width, the improvement in delay is not very significant, but the advantage of reduced critical path is still evident. The waveform confirms correct operation of sum and carry outputs.

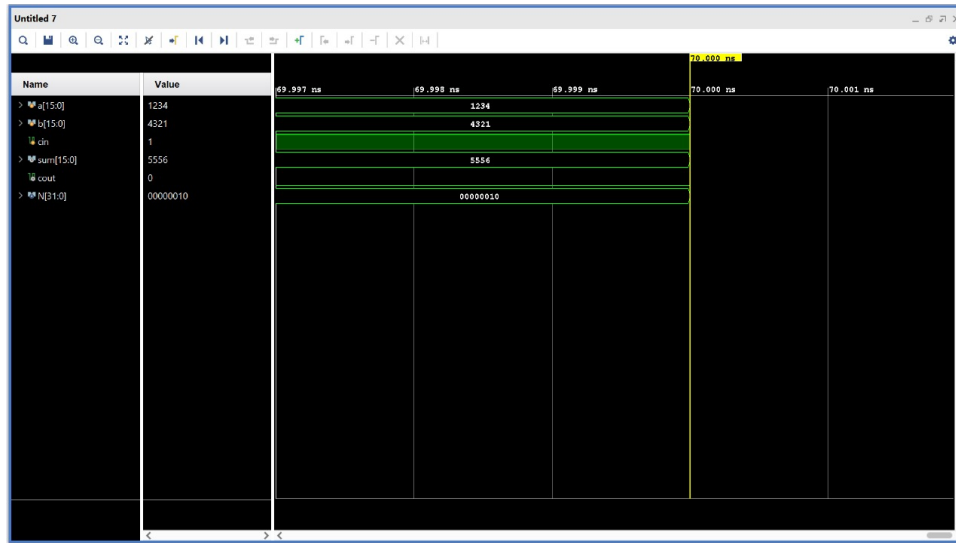


Figure 6.4: Timing diagram of 16 bit HCA.

Figure 6.4 illustrates the timing waveform of the 16 bit Han–Carlson Adder. The use of additional parallel prefix computation reduces the number of carry propagation stages, leading to faster output stabilization. This demonstrates improved speed compared to Brent–Kung, at the cost of slightly increased hardware usage.

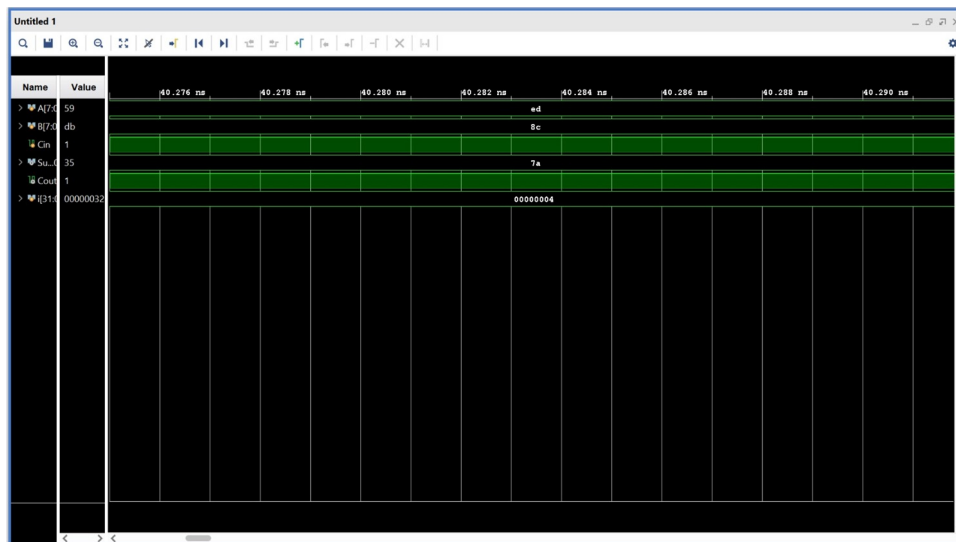


Figure 6.5: Timing diagram of 8 bit HYBRID.

Figure 6.5 shows the timing waveform of the 8-bit Hybrid Adder. At lower bit-widths, the performance difference among architectures is minimal. However, the hybrid structure still achieves slightly faster carry computation due to partial parallelism introduced by the Han–Carlson stages.

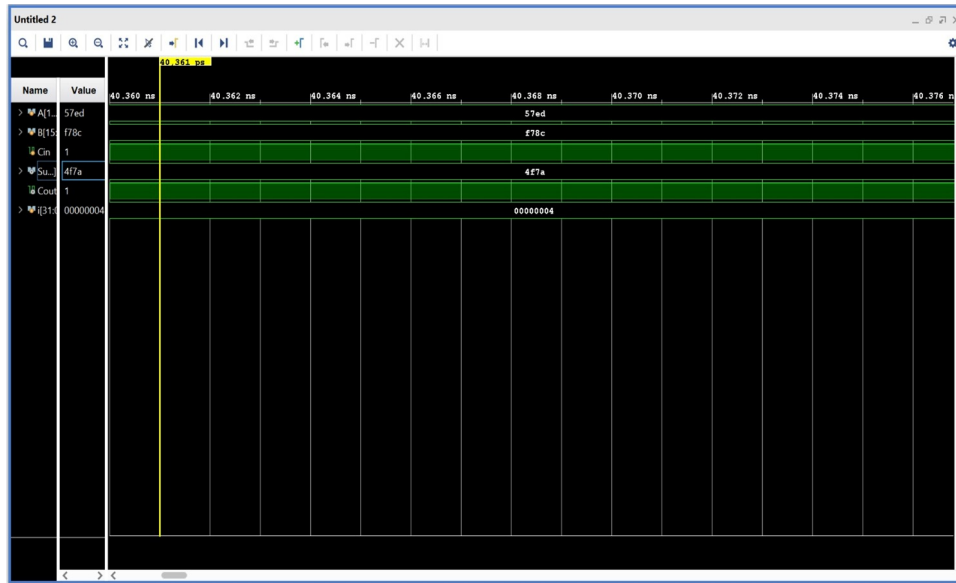


Figure 6.6: Timing diagram of 16 bit HYBRID.

Figure 6.6 presents the timing waveform of the 16-bit Hybrid Adder. A noticeable improvement in delay is observed compared to Brent–Kung. The hybrid structure effectively reduces the critical path by combining efficient prefix computation with controlled hardware usage.

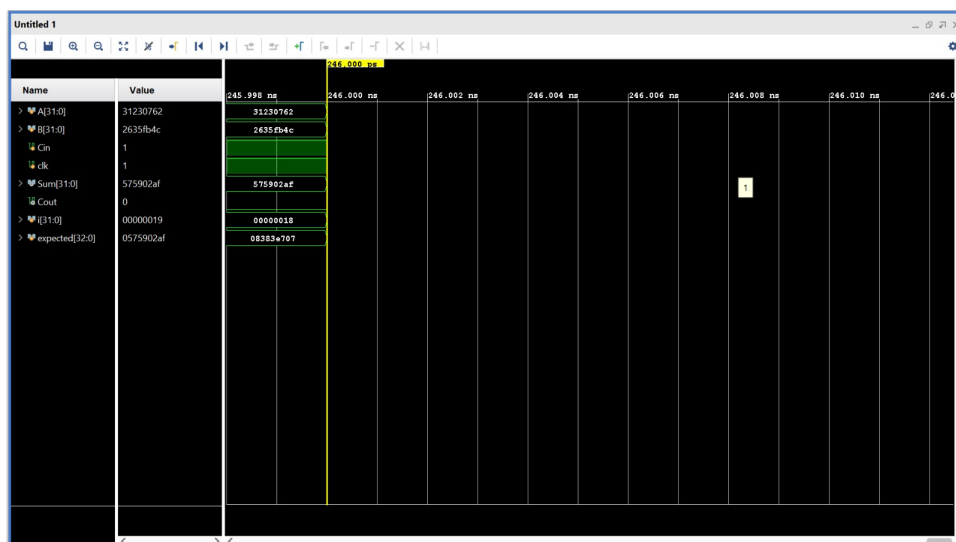


Figure 6.7: Timing diagram of 32 bit HYBRID.

Figure 6.7 shows the timing waveform of the 32-bit Hybrid Adder. The outputs stabilize faster compared to both BKA and HCA, demonstrating improved parallelism in carry propagation. This highlights the advantage of the hybrid approach for higher bit-width designs.

Overall, Through the timing diagrams the Functionality of all three adders were verified.

6.3 PERFORMANCE ANALYSIS

ADDERS	SLICE	DELAY	NETS
BJA-8	10	13.661	97
HCA-8	10	13.67	99
HYBRID-8	9	13.56	97
BJA-16	27	20.73	199
HCA-16	27	20.32	211
HYBRID-16	28	20.07	201
BJA-32	66	34.45	405
HCA-32	72	35.88	451
HYBRID-32	60	34.741	417

Table 6.2: Performance Comparison of BKA, HCA and Hybrid Adders

Table 6.2 presents the performance comparison of Brent–Kung, Han–Carlson, and Hybrid adders in terms of slice utilization, propagation delay, and routing complexity.

For the 8-bit implementation, all three architectures exhibit similar delay values due to the limited number of prefix levels. However, the Hybrid Adder achieves slightly lower delay and reduced slice utilization, indicating efficient resource usage even at smaller scales.

In the 16-bit case, the difference becomes more significant. The Hybrid Adder achieves the lowest delay (20.07 ns), outperforming both BKA and HCA. This improvement is attributed to the balanced combination of prefix structures, which reduces the critical path length without significantly increasing hardware complexity.

For the 32-bit implementation, the advantages of the Hybrid Adder become more prominent. It uses fewer slices compared to both BKA and HCA, demonstrating better area efficiency. In terms of delay, it performs close to the fastest architecture while maintaining lower routing complexity.

The results clearly indicate that the Hybrid Carry Tree Adder provides an optimal trade-off between delay, area, and interconnect complexity, making it suitable for high-speed and large bit-width applications.

The theoretical delay analysis in Table 6.3 is based on the number of black and grey cells in the critical path. The delay of a grey cell is represented as t_g and that of a black cell as t_b . The overall delay is determined by the critical path, which is the longest sequence of prefix operations from input to output.

ADDERS	CRITICAL PATH	DELAY (ns)
KA 8	$4t_g$	26.06
HA 8	$2t_g + 2t_b$	26.6
HYBRID 8	$3t_g + t_b$	26.36
KA 16	$4t_g + 2t_b$	39.69
HA 16	$2t_g + 3t_b$	33.47
HYBRID 16	$3t_g + 2t_b$	33.18
KA 32	$5t_g + 3t_b$	53.03
HA 32	$2t_g + 4t_b$	40.29
HYBRID 32	$3t_g + 3t_b$	39.99

Table 6.3: Theoretical Analysis of Delay

For smaller bit-widths, all architectures show comparable delays due to limited prefix levels. As the bit-width increases, the Hybrid Adder achieves lower delay by optimizing the number of prefix stages and balancing the usage of black and grey cells.

This demonstrates that the hybrid structure effectively minimizes the critical path while avoiding excessive hardware overhead, validating the practical simulation results.

For the 8-bit implementation, all three adders exhibit similar delays. KA has a delay of $4t_g = 26.06$ ns, HA has $2t_g + 2t_b = 26.6$ ns, and the Hybrid Adder has $3t_g + t_b = 26.36$ ns. Since the bit-width is small, the number of stages in the prefix network is limited, resulting in comparable delays for all architectures.

In the 16-bit case, the impact of architecture becomes more noticeable. KA exhibits a higher delay of $4t_g + 2t_b = 39.69$ ns. HA reduces the delay to 33.47 ns by introducing more parallel carry computation. The Hybrid Adder achieves a slightly lower delay of 33.18 ns with $3t_g + 2t_b$, indicating a better balance between grey and black cell usage in the critical path.

For the 32-bit implementation, the difference is significant. KA has the highest delay of 53.03 ns due to its deeper prefix structure. HA reduces the delay to 40.29 ns by increasing parallelism. The Hybrid Adder achieves the lowest delay of 39.99 ns with $3t_g + 3t_b$, demonstrating that it effectively minimizes the critical path by optimally combining grey and black cells.

Overall, the theoretical results show that as bit-width increases, the Hybrid Adder consistently provides a better trade-off between logic depth and parallelism. It achieves delay values close to or better than HA while avoiding the higher complexity of fully parallel structures. Thus, the hybrid architecture proves to be more efficient for higher bit-width designs.

Chapter 7

CONCLUSION

This work presented the design, implementation, and evaluation of high-speed parallel prefix adders, namely the Brent–Kung Adder (BKA), Han–Carlson Adder (HCA), and the proposed Hybrid Carry Tree Adder (HCA–BKA). All architectures were modeled using Verilog HDL, simulated, and synthesized in Xilinx Vivado, and evaluated on FPGA based on key performance metrics such as propagation delay, slice utilization, and routing complexity.

From the experimental results, it is observed that for lower bit-widths (8-bit), all architectures exhibit nearly similar performance due to the limited depth of carry propagation. However, as the bit-width increases to 16-bit and 32-bit, the differences in architectural design become more prominent. The Brent–Kung adder demonstrates lower area and reduced wiring complexity but suffers from higher delay due to increased logic depth. In contrast, the Han–Carlson adder achieves faster carry propagation through increased parallelism, at the expense of higher resource utilization and interconnect complexity.

The proposed Hybrid Carry Tree Adder effectively combines the advantages of both architectures by employing Brent–Kung logic in the lower stages and Han–Carlson logic in the higher stages. This hybrid approach reduces the critical path delay while maintaining moderate hardware utilization. The results show that the Hybrid Adder achieves lower or comparable delay to HCA while using fewer resources, particularly in higher bit-width implementations.

Although minor variations in delay are observed in FPGA-based results, these are primarily due to synthesis and routing optimizations performed by the design tool, rather than limitations of the architecture itself. The theoretical analysis based on prefix structures and the distribution of black and grey cells confirms the expected delay trends, validating the correctness of the proposed design.

Overall, the Hybrid Carry Tree Adder provides an efficient trade-off between speed, area, and wiring complexity, making it a suitable candidate for high-speed and resource-efficient

digital systems.

7.1 Limitations

Despite the improvements achieved, certain limitations exist in the proposed design:

- The performance analysis is limited to FPGA implementation; ASIC-level behavior may differ.
- Power consumption has not been deeply analyzed, which is an important factor in VLSI design.
- Routing delays introduced by FPGA tools may affect the actual critical path, leading to slight variations from theoretical results.
- The hybrid partitioning strategy is fixed and not dynamically optimized for different input conditions.

7.2 Future Work

The proposed work can be further extended in the following directions:

- Implementation of the hybrid adder in ASIC design flow for more accurate performance evaluation.
- Detailed power and energy consumption analysis for low-power applications.
- Optimization of partitioning strategies using adaptive or algorithm-based approaches.
- Exploration of hybrid designs combining other prefix architectures for further performance improvement.
- Integration of the proposed adder into larger systems such as ALUs or DSP processors.

References

- [1] P. Kousalya, M. Lavanya, and T. Archana, "Carry Tree Adders on FPGAs Design and its Implementation," *Proc. IEEE First International Conference for Women in Computing (INCOWOCO)*, 2024.
- [2] S. K. Yezerla and B. Rajendra Naik, "Design and Estimation of Delay, Power and Area for Parallel Prefix Adders," in *Proc. 2014 Recent Advances in Engineering and Computational Sciences (RAECS)*, 2014.
- [3] Anjana D and Jemti Jose "Design and FPGA Implementation of Optimized Parallel Prefix Adder," *IJIRSET*, July 2016.
- [4] Senthil Ganesh R and R. Kalaimathi "Design and Analysis of Kogge-Stone and Han-Carlson Adders in 130nm CMOS Technology," *IJAERD*, March 2018.
- [5] G.Chandrasekhar and Mohammad Taj"Implementation of PPA-Brent Kung Adder for Computing Application," *International Journal of Research*, May 2017.
- [6] Sreenivaas Muthyala Sudhakar, Kumar P. Chidambaram and Earl E. Swartzlander Jr"Hybrid Han-Carlson Adder," *IEEE Symposium on Computer Arithmetic*, 2012.
- [7] Sameen Gauhar, Adeeba Sharif, and Naushad Alam "Comparison of Parallel Prefix Adders Based on FPGA & ASIC Implementations," *IEEE SCES*, 2020.
- [8] D. K. Athur, B. Narayanan, A. Gopalakrishnan, S. Palanisamy, and A. A. Augustine, "Design of novel high speed parallel prefix adder," *Indonesian Journal of Electrical Engineering and Computer Science(IJEECS)*, Mar. 2023.

APPENDIX

.0.1 Black Cell Code

```
1 module black_cell(  
2   input  Gk_j, Pk_j,  
3   input  Gj_i, Pj_i,  
4   output Gk_i, Pk_i  
5 );  
6  
7 assign Gk_i = Gk_j | (Pk_j & Gj_i);  
8 assign Pk_i = Pk_j & Pj_i;  
9 endmodule
```

Listing 1: Verilog code for black cell

.0.2 Grey Cell code

```
1 module grey_cell(  
2   input  Gk_j, Pk_j,  
3   input  Gj_i,  
4   output Gk_i  
5 );  
6  
7 assign Gk_i = Gk_j | (Pk_j & Gj_i);  
8  
9 endmodule
```

Listing 2: Verilog Code for grey cell

.0.3 Source code for 8 bit bka adder

```
1  
2 (* keep_hierarchy = "yes" *)  
3 module bka8_full(  
4   input [7:0] A,  
5   input [7:0] B,  
6   input Cin,
```

```

7  output [7:0] Sum,
8  output Cout
9  );
10
11 ( dont_touch = "true" *)wire [7:0] P,G;
12 assign P = A ^ B;
13 assign G = A & B;
14
15 (* dont_touch = "true" *)wire G10;
16 (* dont_touch = "true" *)wire G20;
17 (* dont_touch = "true" *)wire G32,P32;
18 (* dont_touch = "true" *)wire G30;
19 (* dont_touch = "true" *)wire G40;
20 (* dont_touch = "true" *)wire G54,P54;
21 (* dont_touch = "true" *)wire G50;
22 (* dont_touch = "true" *)wire G60;
23 (* dont_touch = "true" *)wire G76,P76;
24 (* dont_touch = "true" *)wire G74,P74;
25 (* dont_touch = "true" *)wire G70;
26
27 grey_cell gc10(G[1],P[1],G[0],G10);
28
29 black_cell bc32(G[3],P[3],G[2],P[2],G32,P32);
30 black_cell bc54(G[5],P[5],G[4],P[4],G54,P54);
31 black_cell bc76(G[7],P[7],G[6],P[6],G76,P76);
32 grey_cell gc30(G32,P32,G10,G30);
33 black_cell bc74(G76,P76,G54,P54,G74,P74);
34 grey_cell gc70(G74,P74,G30,G70);
35
36 grey_cell gc20(G[2],P[2],G10,G20);
37 grey_cell gc40(G[4],P[4],G30,G40);
38 grey_cell gc50(G54,P54,G30,G50);
39 grey_cell gc60(G[6],P[6],G50,G60);
40
41 (* dont_touch = "true" *)wire P10 = P[1]&P[0];
42 (* dont_touch = "true" *)wire P20 = P[2]&P10;
43 (* dont_touch = "true" *)wire P30 = P[3]&P20;
44 (* dont_touch = "true" *)wire P40 = P[4]&P30;
45 (* dont_touch = "true" *)wire P50 = P[5]&P40;

```



```
46 (* dont_touch = "true" *)wire P60 = P[6]&P50;
47 (* dont_touch = "true" *)wire P70 = P[7]&P60;
48
49 (* keep = "true" *)wire C1,C2,C3,C4,C5,C6,C7,C8;
50 assign C1 = G[0] | (P[0] & Cin);
51 assign C2 = G10 | (P10 & Cin);
52 assign C3 = G20 | (P20 & Cin);
53 assign C4 = G30 | (P30 & Cin);
54 assign C5 = G40 | (P40 & Cin);
55 assign C6 = G50 | (P50 & Cin);
56 assign C7 = G60 | (P60 & Cin);
57 assign C8 = G70 | (P70 & Cin);
58 assign Cout = C8;
59
60 assign Sum[0] = P[0] ^ Cin;
61 assign Sum[1] = P[1] ^ C1;
62 assign Sum[2] = P[2] ^ C2;
63 assign Sum[3] = P[3] ^ C3;
64 assign Sum[4] = P[4] ^ C4;
65 assign Sum[5] = P[5] ^ C5;
66 assign Sum[6] = P[6] ^ C6;
67 assign Sum[7] = P[7] ^ C7;
68 endmodule
```

Listing 3: Verilog Code for 8 bit bka adder

.0.4 Test bench code for 8 bit bka adder

```
1  `timescale 1ns/1ps
2
3  module tb_bka8_full;
4
5  reg [7:0] A, B;
6  reg Cin;
7
8  wire [7:0] Sum;
9  wire Cout;
10
11 // DUT
```

```
12 bka8_full uut (  
13     .A(A),  
14     .B(B),  
15     .Cin(Cin),  
16     .Sum(Sum),  
17     .Cout(Cout)  
18 );  
19  
20 integer i;  
21 reg [8:0] expected;  
22  
23 initial begin  
24     $display("A\tB\tCin\t\tSum\tCout\t\tExpected");  
25  
26     for (i = 0; i < 20; i = i + 1) begin  
27  
28         A = $random;  
29         B = $random;  
30         Cin = $random;  
31  
32         #1;  
33  
34         expected = A + B + Cin;  
35  
36         $display("%h\t%h\t%b\t\t\t%h\t%b\t\t\t%h",  
37             A, B, Cin, Sum, Cout, expected);  
38  
39         if ({Cout, Sum} != expected)  
40             $display("ERROR");  
41         else  
42             $display("CORRECT");  
43     end  
44  
45     $finish;  
46 end  
47  
48 endmodule
```

Listing 4: Test bench code for 8 bit bka adder

.0.5 Source code for 8 bit hca adder

```
1 (* keep_hierarchy = "yes" *)
2 module hca8 (
3   input  [7:0] A,
4   input  [7:0] B,
5   input  Cin,
6   output [7:0] Sum,
7   output Cout
8 );
9
10 (* dont_touch = "true" *)wire [7:0] P,G;
11 assign P = A ^ B;
12 assign G = A & B;
13
14 (* dont_touch = "true" *)wire G10;
15 grey_cell gc10(G[1],P[1],G[0],G10);
16
17 (* dont_touch = "true" *)wire G32,P32;
18 black_cell bc32(G[3],P[3],G[2],P[2],G32,P32);
19
20 (* dont_touch = "true" *)wire G54,P54;
21 black_cell bc54(G[5],P[5],G[4],P[4],G54,P54);
22
23 (* dont_touch = "true" *)wire G76,P76;
24 black_cell bc76(G[7],P[7],G[6],P[6],G76,P76);
25
26 (* dont_touch = "true" *)wire G30;
27 grey_cell gc30(G32,P32,G10,G30);
28
29 (* dont_touch = "true" *)wire G52,P52;
30 black_cell bc52(G54,P54,G32,P32,G52,P52);
31
32 (* dont_touch = "true" *)wire G74,P74;
33 black_cell bc74(G76,P76,G54,P54,G74,P74);
34 (* dont_touch = "true" *)wire G50;
35 grey_cell gc50(G52,P52,G10,G50);
36 (* dont_touch = "true" *)wire G70;
37 grey_cell gc70(G74,P74,G30,G70);
```

```
38 (* dont_touch = "true" *)wire G20;
39 grey_cell gc20(G[2],P[2],G10,G20);
40
41 (* dont_touch = "true" *)wire G40;
42 grey_cell gc40(G[4],P[4],G30,G40);
43 (* dont_touch = "true" *)wire G60;
44 grey_cell gc60(G[6],P[6],G50,G60);
45
46 (* dont_touch = "true" *)wire P10 = P[1] & P[0];
47 (* dont_touch = "true" *)wire P20 = P[2] & P10;
48 (* dont_touch = "true" *)wire P30 = P[3] & P20;
49 (* dont_touch = "true" *)wire P40 = P[4] & P30;
50 (* dont_touch = "true" *)wire P50 = P[5] & P40;
51 (* dont_touch = "true" *)wire P60 = P[6] & P50;
52 (* dont_touch = "true" *)wire P70 = P[7] & P60;
53
54
55 (* keep = "true" *)wire C1,C2,C3,C4,C5,C6,C7,C8;
56 assign C1 = G[0] | (P[0] & Cin);
57 assign C2 = G10 | (P10 & Cin);
58 assign C3 = G20 | (P20 & Cin);
59 assign C4 = G30 | (P30 & Cin);
60 assign C5 = G40 | (P40 & Cin);
61 assign C6 = G50 | (P50 & Cin);
62 assign C7 = G60 | (P60 & Cin);
63 assign C8 = G70 | (P70 & Cin);
64
65 assign Cout = C8;
66
67 assign Sum[0] = P[0] ^ Cin;
68 assign Sum[1] = P[1] ^ C1;
69 assign Sum[2] = P[2] ^ C2;
70 assign Sum[3] = P[3] ^ C3;
71 assign Sum[4] = P[4] ^ C4;
72 assign Sum[5] = P[5] ^ C5;
73 assign Sum[6] = P[6] ^ C6;
74 assign Sum[7] = P[7] ^ C7;
75
76 endmodule
```

Listing 5: Source code: 8 bit hca adder

.0.6 Test bench code for 8 bit hca adder

```
1  'timescale 1ns/1ps
2
3  module tb_hca8;
4
5  reg [7:0] A, B;
6  reg Cin;
7
8  wire [7:0] Sum;
9  wire Cout;
10
11 // DUT
12 hca8 uut (
13     .A(A),
14     .B(B),
15     .Cin(Cin),
16     .Sum(Sum),
17     .Cout(Cout)
18 );
19
20 integer i;
21 reg [8:0] expected;
22
23 initial begin
24     $display("A\tB\tCin\t\tSum\tCout\t\tExpected");
25
26     for (i = 0; i < 20; i = i + 1) begin
27
28         A    = $random;
29         B    = $random;
30         Cin  = $random;
31
32         #1;
33
```

```
34     expected = A + B + Cin;
35
36     $display("%h\t%h\t%b\t|\t%h\t%b\t|\t%h",
37             A, B, Cin, Sum, Cout, expected);
38
39     if ({Cout, Sum} !== expected)
40         $display("ERROR");
41     else
42         $display("CORRECT");
43 end
44
45 $finish;
46 end
47
48 endmodule
```

Listing 6: Test bench Code for 8-bit hca adder

.0.7 Source code for 8 bit hybrid adder

```
1  (* keep_hierarchy = "yes" *)
2  module hybrid8(
3      input [7:0] A,
4      input [7:0] B,
5      input Cin,
6      output [7:0] Sum,
7      output Cout
8  );
9
10 (* dont_touch = "true" *)wire [7:0] P,G;
11 assign P = A ^ B;
12 assign G = A & B;
13
14 (* dont_touch = "true" *)wire G10,P10;
15 (* dont_touch = "true" *)wire G32,P32;
16 (* dont_touch = "true" *)wire G30,P30;
17 (* dont_touch = "true" *)wire G20,P20;
18
19
```

```
20 grey_cell gc10(G[1],P[1],G[0],G10);
21 black_cell bc32(G[3],P[3],G[2],P[2],G32,P32);
22
23 grey_cell gc30(G32,P32,G10,G30);
24 grey_cell gc20(G[2],P[2],G10,G20);
25 assign P10 = P[1] & P[0];
26 assign P20 = P[2] & P10;
27 assign P30 = P[3] & P20;
28
29
30
31 (* dont_touch = "true" *)wire G54,P54;
32 (* dont_touch = "true" *)wire G76,P76;
33 (* dont_touch = "true" *)wire G74,P74;
34
35 black_cell bc54(G[5],P[5],G[4],P[4],G54,P54);
36 black_cell bc76(G[7],P[7],G[6],P[6],G76,P76);
37 black_cell bc74(G76,P76,G54,P54,G74,P74);
38
39
40 (* dont_touch = "true" *)wire G70;
41 (* dont_touch = "true" *)wire G40;
42 (* dont_touch = "true" *)wire G50;
43 (* dont_touch = "true" *)wire G60;
44
45 grey_cell gc70(G74,P74,G30,G70);
46 grey_cell gc50(G54,P54,G30,G50);
47 grey_cell gc40(G[4],P[4],G30,G40);
48 grey_cell gc60(G[6],P[6],G50,G60);
49
50 (* dont_touch = "true" *)wire P40,P60,P50,P70;
51 assign P40=P[4]&P30;
52 assign P50=P[5]&P40;
53 assign P60=P[6]&P50;
54 assign P70=P[7]&P60;
55
56 (* keep = "true" *)wire C0,C1,C2,C3,C4,C5,C6,C7;
57 assign C0 = Cin;
58 assign C1 = G[0] | (P[0] & Cin);
```

```
59 assign C2 = G10 | (P10 & Cin);
60 assign C3 = G20 | (P20 & Cin);
61 assign C4 = G30 | (P30 & Cin);
62 assign C5 = G40 | (P40 & Cin);
63 assign C6 = G50 | (P50 & Cin);
64 assign C7 = G60 | (P60 & Cin);
65 assign Cout = G70|(P70&Cin);
66
67
68
69 assign Sum[0] = P[0] ^ C0;
70 assign Sum[1] = P[1] ^ C1;
71 assign Sum[2] = P[2] ^ C2;
72 assign Sum[3] = P[3] ^ C3;
73 assign Sum[4] = P[4] ^ C4;
74 assign Sum[5] = P[5] ^ C5;
75 assign Sum[6] = P[6] ^ C6;
76 assign Sum[7] = P[7] ^ C7;
77
78 endmodule
```

Listing 7: Verilog Source Code for 8-bit hybrid adder

.0.8 Test bench code: 8 bit hybrid adder

```
1
2 `timescale 1ns/1ps
3
4 module tb_hybrid8;
5
6 reg [7:0] A, B;
7 reg Cin;
8 wire [7:0] Sum;
9 wire Cout;
10
11
12 hybrid8 uut (
13     .A(A),
14     .B(B),
```



```

15     .Cin(Cin),
16     .Sum(Sum),
17     .Cout(Cout)
18 );
19
20
21 integer i;
22 reg [8:0] expected;
23
24 initial begin
25     $display("Time\tA\tB\tCin\t\tSum\tCout\t\tExpected");
26
27
28     A = 0; B = 0; Cin = 0;
29
30
31     for (i = 0; i < 20; i = i + 1) begin
32         @(posedge clk);
33
34         A    = $random;
35         B    = $random;
36         Cin  = $random;
37
38         #1;
39
40         expected = A + B + Cin;
41
42         $display("%0t\t%h\t%h\t%b\t\t\t%h\t%b\t\t\t%h",
43                 $time, A, B, Cin, Sum, Cout, expected);
44
45         if ({Cout, Sum} != expected)
46             $display(" ERROR at time %0t", $time);
47         else
48             $display("CORRECT");
49     end
50
51     $finish;
52 end
53

```

54 `endmodule`

Listing 8: Test bench code: 8 bit hybrid adder

.0.9 Source code for 16 bit hybrid adder

```

1
2 (* keep_hierarchy = "yes" *)
3 module hybrid16(
4     input  [15:0] A,
5     input  [15:0] B,
6     input  Cin,
7     output [15:0] Sum,
8     output Cout
9 );
10
11 (* dont_touch = "true" *)wire [15:0] P,G;
12 assign P = A ^ B;
13 assign G = A & B;
14
15 (* dont_touch = "true" *)wire G10,G32,P32,G54,P54,G76,P76;
16 grey_cell gc10(G[1],P[1],G[0],G10);
17
18 black_cell bc32(G[3],P[3],G[2],P[2],G32,P32);
19 black_cell bc54(G[5],P[5],G[4],P[4],G54,P54);
20 black_cell bc76(G[7],P[7],G[6],P[6],G76,P76);
21
22 (* dont_touch = "true" *)wire G30,G74,P74;
23 grey_cell gc30(G32,P32,G10,G30);
24 black_cell bc74(G76,P76,G54,P54,G74,P74);
25 (* dont_touch = "true" *)wire G70;
26 grey_cell gc70(G74,P74,G30,G70);
27
28 (* dont_touch = "true" *)wire G20,G40,G50,G60;
29 grey_cell gc20(G[2],P[2],G10,G20);
30 grey_cell gc40(G[4],P[4],G30,G40);
31 grey_cell gc50(G54,P54,G30,G50);
32 grey_cell gc60(G[6],P[6],G50,G60);
33

```

```
34
35 (* dont_touch = "true" *)wire G98,P98,G1110,P1110,G1312,P1312,G1514,P1514;
36
37 black_cell bc98(G[9],P[9],G[8],P[8],G98,P98);
38 black_cell bc1110(G[11],P[11],G[10],P[10],G1110,P1110);
39 black_cell bc1312(G[13],P[13],G[12],P[12],G1312,P1312);
40 black_cell bc1514(G[15],P[15],G[14],P[14],G1514,P1514);
41
42 (* dont_touch = "true" *)wire G118,P118,G1310,P1310,G1512,P1512;
43
44
45 black_cell bc118(G1110,P1110,G98,P98,G118,P118);
46 black_cell bc1310(G1312,P1312,G1110,P1110,G1310,P1310);
47 black_cell bc1512(G1514,P1514,G1312,P1312,G1512,P1512);
48
49 (* dont_touch = "true" *)wire G158,P158;
50 black_cell bc158(G1512,P1512,G118,P118,G158,P158);
51
52 (* dont_touch = "true" *)wire G150;
53 grey_cell gc150(G158,P158,G70,G150);
54
55 (* dont_touch = "true" *)wire G80,G90,G100,G110,G120,G130,G140;
56 grey_cell gc80(G[8],P[8],G70,G80);
57 grey_cell gc90(G98,P98,G70,G90);
58 grey_cell gc100(G[10],P[10],G90,G100);
59 grey_cell gc110(G118,P118,G70,G110);
60 grey_cell gc120(G[12],P[12],G110,G120);
61 grey_cell gc130(G1310,P1310,G90,G130);
62 grey_cell gc140(G[14],P[14],G130,G140);
63
64 (* dont_touch = "true" *)wire P10,P20,P30,P40,P50,P60,P70,P80,P90,P100,P110
    ,P120,P130,P140,P150;
65 assign P10 = P[1] & P[0];
66 assign P20 = P[2] & P10;
67 assign P30 = P[3] & P20;
68 assign P40 = P[4] & P30;
69 assign P50 = P[5] & P40 ;
70 assign P60 = P[6] & P50;
71 assign P70 = P[7] & P60;
```

```
72 assign P80 = P[8] & P70;
73 assign P90 = P[9] & P80;
74 assign P100 = P[10] & P90;
75 assign P110 = P[11] & P100;
76 assign P120 = P[12] & P110;
77 assign P130 = P[13] & P120;
78 assign P140 = P[14] & P130;
79 assign P150 = P[15] & P140 ;
80
81
82
83 (* keep = "true" *)wire [15:0] C;
84
85 assign C[0] = Cin;
86
87 assign C[1] = G[0] | (P[0] & Cin);
88 assign C[2] = G10 | (P10 & Cin);
89 assign C[3] = G20 | (P20 & Cin);
90 assign C[4] = G30 | (P30 & Cin);
91 assign C[5] = G40 | (P40 & Cin);
92 assign C[6] = G50 | (P50 & Cin);
93 assign C[7] = G60 | (P60 & Cin);
94 assign C[8] = G70 | (P70 & Cin);
95 assign C[9] = G80 | (P80 & Cin);
96 assign C[10] = G90 | (P90 & Cin);
97 assign C[11] = G100 | (P100 & Cin);
98 assign C[12] = G110 | (P110 & Cin);
99 assign C[13] = G120 | (P120 & Cin);
100 assign C[14] = G130 | (P130 & Cin);
101 assign C[15] = G140 | (P140 & Cin);
102 (* keep = "true" *)assign Cout = G150 | (P150 & Cin);
103
104
105 (* keep = "true" *)assign Sum[0] = P[0] ^ C[0];
106 (* keep = "true" *)assign Sum[1] = P[1] ^ C[1];
107 (* keep = "true" *)assign Sum[2] = P[2] ^ C[2];
108 (* keep = "true" *)assign Sum[3] = P[3] ^ C[3];
109 (* keep = "true" *)assign Sum[4] = P[4] ^ C[4];
110 (* keep = "true" *)assign Sum[5] = P[5] ^ C[5];
```

```
111 (* keep = "true" *)assign Sum[6]  = P[6]  ^ C[6];
112 (* keep = "true" *)assign Sum[7]  = P[7]  ^ C[7];
113 (* keep = "true" *)assign Sum[8]  = P[8]  ^ C[8];
114 (* keep = "true" *)assign Sum[9]  = P[9]  ^ C[9];
115 (* keep = "true" *)assign Sum[10] = P[10] ^ C[10];
116 (* keep = "true" *)assign Sum[11] = P[11] ^ C[11];
117 (* keep = "true" *)assign Sum[12] = P[12] ^ C[12];
118 (* keep = "true" *)assign Sum[13] = P[13] ^ C[13];
119 (* keep = "true" *)assign Sum[14] = P[14] ^ C[14];
120 (* keep = "true" *)assign Sum[15] = P[15] ^ C[15];
121
122 endmodule
```

Listing 9: Verilog Source Code for 16-bit hybrid adder

.0.10 Test bench code: 16 bit hybrid adder

```
1
2 `timescale 1ns/1ps
3
4 module tb_hybrid16;
5
6 reg [15:0] A, B;
7 reg Cin;
8 wire [15:0] Sum;
9 wire Cout;
10
11
12 hybrid16 uut (
13     .A(A),
14     .B(B),
15     .Cin(Cin),
16     .Sum(Sum),
17     .Cout(Cout)
18 );
19
20
21 integer i;
22 reg [16:0] expected;
```

```

23
24 initial begin
25     $display("Time\tA\tB\tCin\t\tSum\tCout\t\tExpected");
26
27
28     A = 0; B = 0; Cin = 0;
29
30
31     for (i = 0; i < 20; i = i + 1) begin
32         @(posedge clk);
33
34         A    = $random;
35         B    = $random;
36         Cin  = $random;
37
38         #1;
39
40         expected = A + B + Cin;
41
42         $display("%0t\t%h\t%h\t%b\t\t%h\t%b\t\t%h",
43                 $time, A, B, Cin, Sum, Cout, expected);
44
45         if ({Cout, Sum} !== expected)
46             $display(" ERROR at time %0t", $time);
47         else
48             $display("CORRECT");
49     end
50
51     $finish;
52 end
53
54 endmodule

```

Listing 10: Test bench code: 16 bit hybrid adder

.0.11 Source Code: 32 bit hybrid adder

```

1  (* keep_hierarchy = "yes" *)
2  module hybrid32(

```

```
3 input [31:0] A,
4 input [31:0] B,
5 input Cin,
6 output [31:0] Sum,
7 output Cout
8 );
9
10 (* dont_touch = "true" *)wire [31:0] P,G;
11 assign P = A ^ B;
12 assign G = A & B;
13
14 (* dont_touch = "true" *)wire G10,G32,P32,G54,P54,G76,P76,G98,P98,G1110,
    P1110,G1312,P1312,G1514,P1514;
15
16 grey_cell gc10(G[1],P[1],G[0],G10);
17 black_cell bc32(G[3],P[3],G[2],P[2],G32,P32);
18 black_cell bc54(G[5],P[5],G[4],P[4],G54,P54);
19 black_cell bc76(G[7],P[7],G[6],P[6],G76,P76);
20 black_cell bc98(G[9],P[9],G[8],P[8],G98,P98);
21 black_cell bc1110(G[11],P[11],G[10],P[10],G1110,P1110);
22 black_cell bc1312(G[13],P[13],G[12],P[12],G1312,P1312);
23 black_cell bc1514(G[15],P[15],G[14],P[14],G1514,P1514);
24
25 (* dont_touch = "true" *)wire G30,G74,P74,G118,P118,G1512,P1512;
26
27 grey_cell gc30(G32,P32,G10,G30);
28 black_cell bc74(G76,P76,G54,P54,G74,P74);
29 black_cell bc118(G1110,P1110,G98,P98,G118,P118);
30 black_cell bc1512(G1514,P1514,G1312,P1312,G1512,P1512);
31
32 (* dont_touch = "true" *)wire G70,G158,P158;
33
34 grey_cell gc70(G74,P74,G30,G70);
35 black_cell bc158(G1512,P1512,G118,P118,G158,P158);
36
37 (* dont_touch = "true" *)wire G150;
38 grey_cell gc150(G158,P158,G70,G150);
39
40
```

```
41 (* dont_touch = "true" *)wire G20,G40,G50,G60,G80,G90,G100,G110,G120,G130,
    G140;
42
43 grey_cell gc20(G[2],P[2],G10,G20);
44 grey_cell gc40(G[4],P[4],G30,G40);
45 grey_cell gc50(G54,P54,G30,G50);
46 grey_cell gc60(G[6],P[6],G50,G60);
47
48 grey_cell gc80(G[8],P[8],G70,G80);
49 grey_cell gc90(G98,P98,G70,G90);
50 grey_cell gc100(G[10],P[10],G90,G100);
51 grey_cell gc110(G118,P118,G70,G110);
52 grey_cell gc120(G[12],P[12],G110,G120);
53 grey_cell gc130(G1312,P1312,G110,G130);
54 grey_cell gc140(G[14],P[14],G130,G140);
55
56 (* dont_touch = "true" *)wire G1716,P1716,G1918,P1918,G2120,P2120,G2322,
    P2322;
57 (* dont_touch = "true" *)wire G2524,P2524,G2726,P2726,G2928,P2928,G3130,
    P3130;
58
59 black_cell bc1716(G[17],P[17],G[16],P[16],G1716,P1716);
60 black_cell bc1918(G[19],P[19],G[18],P[18],G1918,P1918);
61 black_cell bc2120(G[21],P[21],G[20],P[20],G2120,P2120);
62 black_cell bc2322(G[23],P[23],G[22],P[22],G2322,P2322);
63 black_cell bc2524(G[25],P[25],G[24],P[24],G2524,P2524);
64 black_cell bc2726(G[27],P[27],G[26],P[26],G2726,P2726);
65 black_cell bc2928(G[29],P[29],G[28],P[28],G2928,P2928);
66 black_cell bc3130(G[31],P[31],G[30],P[30],G3130,P3130);
67
68 (* dont_touch = "true" *)wire G1916,P1916,G2320,P2320,G2724,P2724,G3128,
    P3128;
69
70 black_cell bc1916(G1918,P1918,G1716,P1716,G1916,P1916);
71 black_cell bc2118(G2120,P2120,G1918,P1918,G2118,P2118);
72 black_cell bc2320(G2322,P2322,G2120,P2120,G2320,P2320);
73 black_cell bc2522(G2524,P2524,G2322,P2322,G2522,P2522);
74 black_cell bc2724(G2726,P2726,G2524,P2524,G2724,P2724);
75 black_cell bc2926(G2928,P2928,G2726,P2726,G2926,P2926);
```



```
76 black_cell bc3128(G3130,P3130,G2928,P2928,G3128,P3128);
77
78 (* dont_touch = "true" *)wire G2316,P2316,G2720,P2720,G3124,P3124,P3116,
    G3116;
79
80 black_cell bc2316(G2320,P2320,G1916,P1916,G2316,P2316);
81 black_cell bc2518(G2522,P2522,G2118,P2118,G2518,P2518);
82 black_cell bc2720(G2724,P2724,G2320,P2320,G2720,P2720);
83 black_cell bc2922(G2926,P2926,G2522,P2522,G2922,P2922);
84 black_cell bc3124(G3128,P3128,G2724,P2724,G3124,P3124);
85 black_cell bc3116(G3124,P3124,G2316,P2316,G3116,P3116);
86
87 (* dont_touch = "true" *)wire G310;
88
89 grey_cell gc310(G3116,P3116,G150,G310);
90
91
92 (* dont_touch = "true" *)wire G160,G170,G180,G190,G200,G210,G220,G230,G240,
    G250,G270,G260,G280,G290,G300;
93
94 grey_cell gc160(G[16],P[16],G150,G160);
95 grey_cell gc170(G1716,P1716,G150,G170);
96 grey_cell gc180(G[18],P[18],G170,G180);
97 grey_cell gc190(G1916,P1916,G150,G190);
98 grey_cell gc200(G[20],P[20],G190,G200);
99 grey_cell gc210(G2118,P2118,G170,G210);
100 grey_cell gc220(G[22],P[22],G210,G220);
101 grey_cell gc230(G2316,P2316,G150,G230);
102 grey_cell gc240(G[24],P[24],G230,G240);
103 grey_cell gc250(G2518,P2518,G170,G250);
104 grey_cell gc260(G[26],P[26],G250,G260);
105 grey_cell gc270(G2720,P2720,G190,G270);
106 grey_cell gc280(G[28],P[28],G270,G280);
107 grey_cell gc290(G2922,P2922,G210,G290);
108 grey_cell gc300(G[30],P[30],G290,G300);
109
110 (* dont_touch = "true" *)wire P10,P20,P30,P40,P50,P60,P70;
111 (* dont_touch = "true" *)wire P80,P90,P100,P110,P120,P130,P140,P150;
112 (* dont_touch = "true" *)wire P160,P170,P180,P190,P200,P210,P220,P230;
```

```
113 (* dont_touch = "true" *)wire P240,P250,P260,P270,P280,P290,P300,P310;
114
115 assign P10  = P[1]  & P[0];
116 assign P20  = P[2]  & P10;
117 assign P30  = P[3]  & P20;
118
119 assign P40  = P[4]  & P30;
120 assign P50  = P[5]  & P40;
121 assign P60  = P[6]  & P50;
122 assign P70  = P[7]  & P60;
123
124 assign P80  = P[8]  & P70;
125 assign P90  = P[9]  & P80;
126 assign P100 = P[10] & P90;
127 assign P110 = P[11] & P100;
128
129 assign P120 = P[12] & P110;
130 assign P130 = P[13] & P120;
131 assign P140 = P[14] & P130;
132 assign P150 = P[15] & P140;
133
134 assign P160 = P[16] & P150;
135 assign P170 = P[17] & P160;
136 assign P180 = P[18] & P170;
137
138 assign P190 = P[19] & P180;
139 assign P200 = P[20] & P190;
140 assign P210 = P[21] & P200;
141
142 assign P220 = P[22] & P210;
143 assign P230 = P[23] & P220;
144
145 assign P240 = P[24] & P230;
146 assign P250 = P[25] & P240;
147 assign P260 = P[26] & P250;
148
149 assign P270 = P[27] & P260;
150 assign P280 = P[28] & P270;
151 assign P290 = P[29] & P280;
```

```

152
153 assign P300 = P[30] & P290;
154 assign P310 = P[31] & P300;
155 (* keep = "true" *)wire [31:0] C;
156 assign C[0] = Cin;
157
158 assign C[1] = G[0] | (P[0] & Cin);
159 assign C[2] = G10 | (P10 & Cin);
160 assign C[3] = G20 | (P20 & Cin);
161 assign C[4] = G30 | (P30 & Cin);
162 assign C[5] = G40 | (P40 & Cin);
163 assign C[6] = G50 | (P50 & Cin);
164 assign C[7] = G60 | (P60 & Cin);
165 assign C[8] = G70 | (P70 & Cin);
166
167 assign C[9] = G80 | (P80 & Cin);
168 assign C[10] = G90 | (P90 & Cin);
169 assign C[11] = G100 | (P100 & Cin);
170 assign C[12] = G110 | (P110 & Cin);
171 assign C[13] = G120 | (P120 & Cin);
172 assign C[14] = G130 | (P130 & Cin);
173 assign C[15] = G140 | (P140 & Cin);
174
175 assign C[16] = G150 | (P150 & Cin);
176 assign C[17] = G160 | (P160 & Cin);
177 assign C[18] = G170 | (P170 & Cin);
178 assign C[19] = G180 | (P180 & Cin);
179
180 assign C[20] = G190 | (P190 & Cin);
181 assign C[21] = G200 | (P200 & Cin);
182 assign C[22] = G210 | (P210 & Cin);
183 assign C[23] = G220 | (P220 & Cin);
184 assign C[24] = G230 | (P230 & Cin);
185
186 assign C[25] = G240 | (P240 & Cin);
187 assign C[26] = G250 | (P250 & Cin);
188 assign C[27] = G260 | (P260 & Cin);
189 assign C[28] = G270 | (P270 & Cin);
190 assign C[29] = G280 | (P280 & Cin);

```

```
191 assign C[30] = G290 | (P290 & Cin);
192 assign C[31] = G300 | (P300 & Cin);
193
194 assign Cout = G310 | (P310 & Cin);
195
196 assign Sum[0] = P[0] ^ C[0];
197 assign Sum[1] = P[1] ^ C[1];
198 assign Sum[2] = P[2] ^ C[2];
199 assign Sum[3] = P[3] ^ C[3];
200 assign Sum[4] = P[4] ^ C[4];
201 assign Sum[5] = P[5] ^ C[5];
202 assign Sum[6] = P[6] ^ C[6];
203 assign Sum[7] = P[7] ^ C[7];
204 assign Sum[8] = P[8] ^ C[8];
205 assign Sum[9] = P[9] ^ C[9];
206 assign Sum[10] = P[10] ^ C[10];
207 assign Sum[11] = P[11] ^ C[11];
208 assign Sum[12] = P[12] ^ C[12];
209 assign Sum[13] = P[13] ^ C[13];
210 assign Sum[14] = P[14] ^ C[14];
211 assign Sum[15] = P[15] ^ C[15];
212
213 assign Sum[16] = P[16] ^ C[16];
214 assign Sum[17] = P[17] ^ C[17];
215 assign Sum[18] = P[18] ^ C[18];
216 assign Sum[19] = P[19] ^ C[19];
217 assign Sum[20] = P[20] ^ C[20];
218 assign Sum[21] = P[21] ^ C[21];
219 assign Sum[22] = P[22] ^ C[22];
220 assign Sum[23] = P[23] ^ C[23];
221 assign Sum[24] = P[24] ^ C[24];
222 assign Sum[25] = P[25] ^ C[25];
223 assign Sum[26] = P[26] ^ C[26];
224 assign Sum[27] = P[27] ^ C[27];
225 assign Sum[28] = P[28] ^ C[28];
226 assign Sum[29] = P[29] ^ C[29];
227 assign Sum[30] = P[30] ^ C[30];
228 assign Sum[31] = P[31] ^ C[31];
229 endmodule
```

Listing 11: Source Code for 32-bit hybrid adder

.0.12 Test bench code: 32 bit hybrid adder

```
1
2  `timescale 1ns/1ps
3
4  module tb_hybrid32;
5
6  reg [31:0] A, B;
7  reg Cin;
8  wire [31:0] Sum;
9  wire Cout;
10
11
12  hybrid32 uut (
13      .A(A),
14      .B(B),
15      .Cin(Cin),
16      .Sum(Sum),
17      .Cout(Cout)
18  );
19
20
21  integer i;
22  reg [32:0] expected;
23
24  initial begin
25      $display("Time\tA\tB\tCin\t\tSum\tCout\t\tExpected");
26
27
28      A = 0; B = 0; Cin = 0;
29
30      for (i = 0; i < 25; i = i + 1) begin
31          @(posedge clk);
32
33          A = $random;
```

```
34      B    = $random;
35      Cin = $random;
36
37      #1;
38
39      expected = A + B + Cin;
40
41      $display("%0t\t%h\t%h\t%b\t|\t%h\t%b\t|\t%h",
42              $time, A, B, Cin, Sum, Cout, expected);
43
44      if ({Cout, Sum} != expected)
45          $display(" ERROR at time %0t", $time);
46      else
47          $display("CORRECT");
48      end
49
50      $finish;
51  end
52
53  endmodule
```

Listing 12: Test bench code: 32 bit hybrid adder